Contents

1 PWact 主动学习 4
1.1 预训练数据制备 5
1.2 主动学习 6
1.3 依赖应用 6
1.4 安装流程 7
1.4.1 pip 命令安装 7
1.4.2 github 源码安装 7
1.5 命令列表 8
1.5.1 输出可用命令列表 8
1.5.2 初始训练集制备 8
1.5.3 主动学习 8
1.5.4 gather_pwdata 命令 8
1.5.5 kill 命令 9
1.5.6 filter 命令 9
1.6 输入文件 9
1.6.1 输入文件 param.json 10
1.6.2 输入文件 resource.json 10
1.7 主动学习案例 10
1.7.1 硅的主动学习 10
1.7.2 硅的主动学习 (bigmodel+direct) 10
1.7.3 金银合金的主动学习 10

2 init_bulk param.json 10
2.1 data_format 10
2.2 reserve_work 10
2.3 interval 11
2.4 sys_config_prefix 11
2.5 sys_configs 11
2.5.1 config 11
2.5.2 format 11
2.5.3 relax 11
2.5.4 relax_input_idx 11
2.5.5 super_cell 12
2.5.6 scale 12
2.5.7 perturb 12
2.5.8 cell_pert_fraction 12
2.5.9 atom_pert_distance 12
2.5.10 aimd 12

2.5.11 aimd_input_idx . . . 12
2.5.12 bigmodel . . . 12
2.5.13 bigmodel_input_idx . . . 13
2.5.14 direct . . . 13
2.6 sys_config 设置例子 . . . 13
2.7 dft_style . . . 14
2.8 pseudo . . . 14
2.9 gaussian_param . . . 14
2.10 relax_input . . . 14
2.11 aimd_input . . . 14
2.11.1 input . . . 15
2.11.2 kspacing . . . 15
2.11.3 flag_symm . . . 15
2.12 bigmodel_input . . . 15
2.13 direct_input . . . 15
2.14 完整例子 . . . 15
2.14.1 上述参数作用 . . . 16
2.14.2 relax_input 和 aimd_input . . . 17
2.14.3 pwmat 设置例子 . . . 17
2.14.4 vasp 设置例子 . . . 19
2.14.5 cp2k 设置例子 . . . 19

3 run_iter param.json 19

4 param.json 19
4.1 reserve_work . . . 19
4.2 reserve_md_traj . . . 20
4.3 reserve_scf_files . . . 20
4.4 data_format . . . 20
4.5 init_data . . . 20
4.6 valid_data . . . 20
4.7 init_model_list . . . 20
4.8 use_pre_model . . . 20
4.9 train . . . 21
4.9.1 train_input_file . . . 21
4.10 strategy . . . 22
4.10.1 uncertainty . . . 22
4.10.2 lmps_tolerance . . . 22
4.10.3 lower_model_deiv_f . . . 22
4.10.4 upper_model_deiv_f . . . 22
4.10.5 model_num . . . 23

4.10.6 max_select . . . 23
4.10.7 direct . . . 23
4.10.8 direct_script . . . 23
4.10.9 compress . . . 23
4.10.10 compress_order . . . 23
4.10.11 Compress_dx . . . 23
4.10.12 例子 . . . 24
5.1 explore . . . 24
5.1.1 sys_config_prefix . . . 24
5.1.2 sys_configs . . . 24
5.1.3 lmps_prefix . . . 25
5.1.4 lmps_in . . . 25
5.1.5 md_jobs . . . 26
5.1.6 dft_style . . . 32
5.1.7 bigmodel_script . . . 32
5.1.8 input . . . 33
5.1.9 kspacing . . . 33
5.1.10 flag_symm . . . 33
5.1.11 pseudo . . . 33
5.1.12 gaussian_param . . . 33
5.1.13 DFT 设置例子 . . . 33
5.1.14 案例参考 . . . 34

6 resource.json 37
6.1 resource.json . . . 37
6.2 参数细节 . . . 40
6.2.1 group_size . . . 41
6.2.2 number_node . . . 41
6.2.3 gpu_per_node . . . 41
6.2.4 cpu_per_node . . . 41
6.2.5 queue_name . . . 41
6.2.6 custom_flags . . . 42
6.2.7 source_list . . . 42
6.2.8 module_list . . . 42
6.2.9 env_list . . . 43
6.3 配置案例详解 . . . 43
6.3.1 train 模块 . . . 43
6.3.2 explore 模块 . . . 45
6.3.3 DFT 模块 . . . 46

7 Example for Si active learning 46
7.1 预训练数据制备 INIT_BULK . . . 47
7.1.1 启动命令 . . . 47
7.1.2 INIT_BULK 目录结构 . . . 47
7.1.3 collection 目录 . . . 47
7.2 主动学习 run . . . 49
7.2.1 启动命令 . . . 49
7.2.2 主动学习文件目录 . . . 49
7.2.3 00.train 目录 . . . 50
7.2.4 01.explore 目录 . . . 51
7.2.5 02.label 目录 . . . 52

8 Example for Si active learning (Bigmodel and Direct) 55
8.1 预训练数据制备 INIT_BULK . . . 55
8.1.1 启动命令 . . . 55
8.1.2 INIT_BULK 目录结构 . . . 55
8.1.3 接入 大模型 MD . . . 56
8.1.4 接入 direct 采样 . . . 57
8.2 主动学习 run . . . 57
8.2.1 主动学习文件目录 . . . 57
8.2.2 train 目录 . . . 57
8.2.3 explore 目录 . . . 58
8.2.4 label 目录 . . . 58
8.2.5 接入 大模型 标记 . . . 58
8.2.6 result . . . 58

9 Example for AuAg active learning 59
9.1 预训练数据制备 INIT_BULK . . . 59
9.1.1 启动命令 . . . 59
9.1.2 INIT_BULK 目录结构 . . . 59
9.2 主动学习 run . . . 59
9.2.1 主动学习文件目录 . . . 59

1 PWact 主动学习

机器学习力场相比于传统方法，能够更快和精确地预测材料性质和反应机理，当前最先进的基于深度学习的分子动力学已经能够做到百亿原子体系的模拟。但是由于机器学习方法的插值特性，对于训练集之外的相空间，MLFF 很难做出准确预测。由于训练数据通常是使用昂贵的第一性原理计算生成的，现实中很难获取到大量的从头算数据集，生成具有足够代表性的训练数据但不依赖大量从头算数据，对于提升模型的外推能力

至关重要。PWact (Active learning based on PWmat Machine Learning Force Field) 是开源的基于 MatPL 的一套自动化主动学习平台，用于高效的数据采样。

PWact 平台包含主任务和任务分发器两部分，如 结构图 所示。

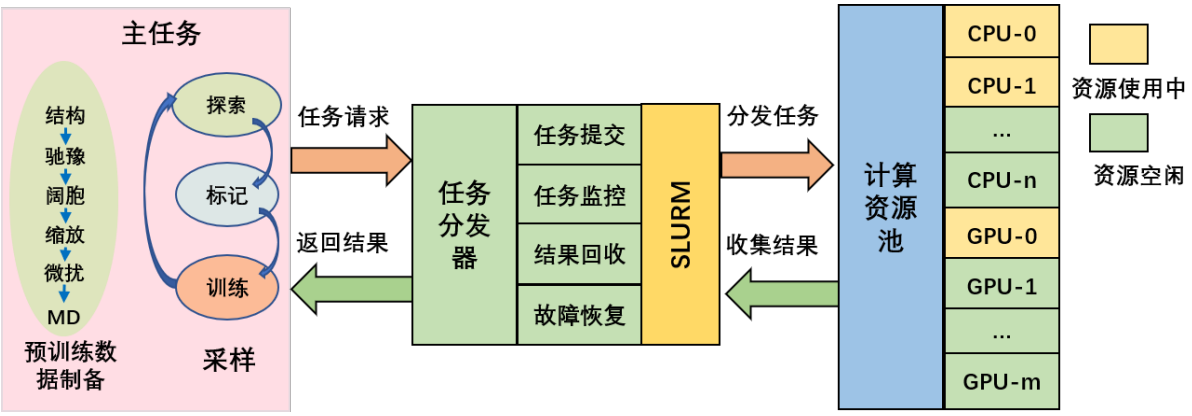

Figure 1: Arch_design_slurm

主任务包括 预训练数据制备 以及 主动学习 两个模块。负责预训练数据制备和主动学习过程中的计算任务生成、以及结果收集。任务分发器接收到任务调度请求之后，根据计算资源使用状态以及任务申请资源情况将任务调度到对应的计算节点上，待任务执行完毕之后，收集计算节点的执行结果返回给主任务程序。

1.1 预训练数据制备

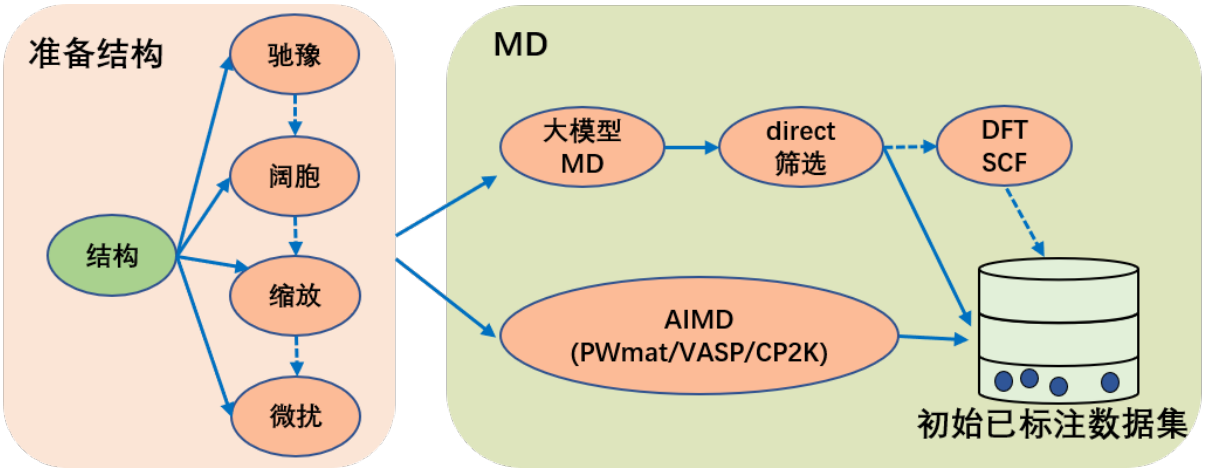

Figure 2: 预训练数据制备流程

预训练数据制备流程包括驰豫（支持 PWmat、VASP、CP2K）、阔胞、缩放晶格、微扰以及运行 MD 四个模块，并支持对这些模块的组合使用。对于 MD，（支持 PWmat、VASP、CP2K）的 AIMD 作为预训练集。同时，也支持使用大模型运行 MD，之后通过 direct 采样过滤掉相似构型后作为预训练集，这里同样支持使用 PWmat、VASP、CP2K 对过滤后的结构做标注（自洽计算获取能量和受力）。

- direct 采样：Qi J, Ko T W, Wood B C, et al. Robust training of machine learning interatomic potentials with dimensionality reduction and stratified sampling[J]. npj Computational Materials, 2024, 10(1): 43.

1.2 主动学习

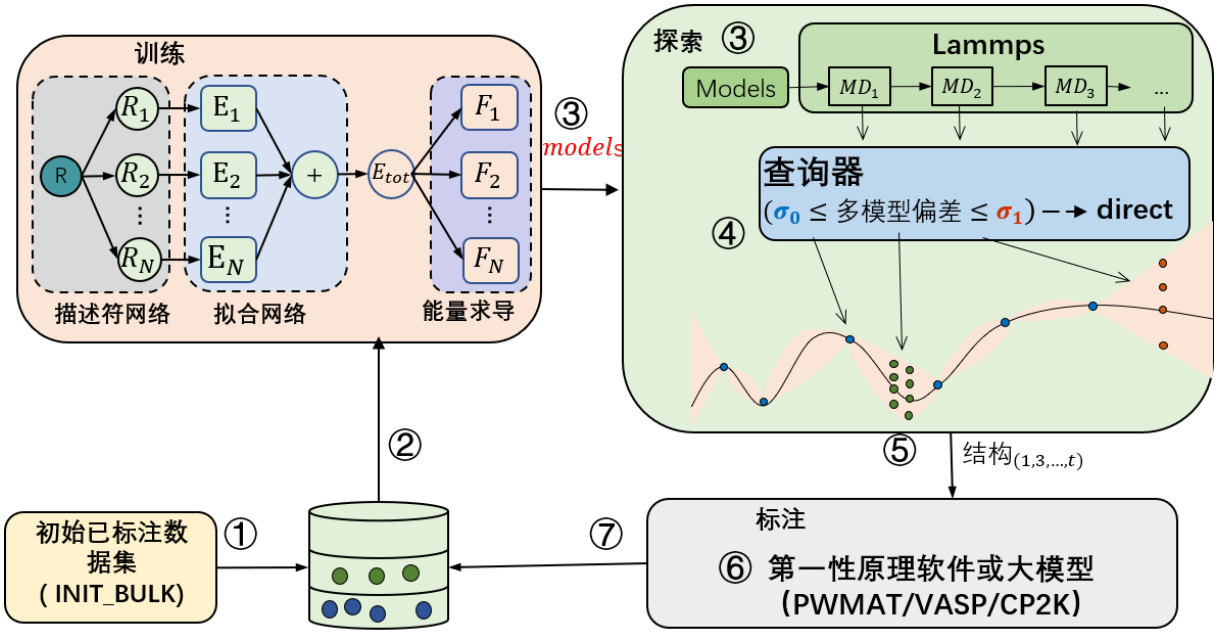

Figure 3: 主动学习流程

主动学习流程包括训练、构型探索以及标注模块。首先，训练模块做模型训练；之后将训练好的模型送入探索模块。探索模块调用力场模型做分子动力学模拟，模拟结束后把得到的分子运动轨迹送入查询器做不确定性度量；查询完成后，把待标注构型点送入标注模块；最后标注模块做自洽计算，得到能量和力，作为标签与对应构型一起送入已标注数据库中；重复上述步骤，直到收敛。

- 对于模型训练，我们这里支持 MatPL 中的 DP model、DP model with compress 以及 DP model with type embedding，以及 NEP 模型。
- 对于不确定性度量，提供了常用了基于多模型委员会查询的方法，并且也提供了我们最新设计的 单模型的基于卡尔曼滤波的不确定性度量方法 KPU (Kalman Prediction Uncertainty, KPU)。该方法能够在接近委员会查询精度的情况下，将模型训练的计算开销减少到 1/N, N 为委员会查询模型数量，欢迎用户尝试。对于 KPU 方法，仅适用于 DP 模型。
- 【可选项】通过 direct 采样方法 降低候选集中的相似结构数量。
- 对于标注，支持 PWmat、VASP、CP2K 或 使用大模型推理。

1.3 依赖应用

- PWact 作业调度采用 SLURM 集群管理和作业调度系统，需要您的计算集群上已安装 SLURM。

- PWact 的 DFT 计算支持 PWmat 、VASP、CP2K，需要您的计算机群已安装 PWMAT、VASP 或 CP2K。
- PWact 使用的力场模型使用 MatPL训练, MatPL 安装方式参考 MatPL 在线手册。
- PWact Lammps 分子动力学模拟 基于 lammps-MatPL，安装方式参考 MatPL 在线手册。
- Direct 采样和大模型采样，目前在 龙讯超算云(Mcloud) 做了预装，支持 direct 采样和 eqv2 大模型。对于用户自己的环境，需要自己安装相应环境，并提供处理脚本。

1.4 安装流程

PWact 支持 pip 命令安装与源码安装两种安装方式。

1.4.1 pip 命令安装

安装包已上传至 pypi 官网，支持直接使用 pip install 安装。

```
pip install pwact

# 安装 pwact, 如果已安装，则升级到最新版本
pip install pwact --upgrade

# 列出所有可安装版本
pip index versions pwact
# 输出结果示例:
# pwact (0.1.27)
# Available versions: 0.1.27, 0.1.25, 0.1.23, 0.1.10
# 安装指定版本
pip install pwact==n.m.o
```

1.4.2 github 源码安装

源码安装适用于需要修改源码的用户，否则建议您通过 pip 安装 即可。源码下载

```
git clone https://github.com/LonxunQuantum/PWact.git
或者
git clone https://gitee.com/pfsuo/pwact.git
gitee 更新可能没有 github 及时，建议优先从 github 下载
```

源码下载后，进入源码的根目录（与 setup.py 同一级）执行命令

```
pip install .
# 或者加开发者选项, 安装时不复制文件，而是直接从源码文件读取，任何对源码的修改都会立即生效
# pip install -e .
```

```
# 从源码安装执行完毕后，需要把 pwact 加入环境变量
# export PYTHONPATH=/the/path/pwact:$PYTHONPATH
```

PWact 开发语言采用 Python，支持 Python 3.9 以及以上的版本。建议用户直接使用 MatPL 的 Python 运行环境 即可。

如果您需要为 PWact 单独创建虚拟环境，只需要安装以下依赖包即可（与您的 Python 版本相匹配，支持 Python 3.9 以及以上）。

```
pip install numpy pandas tqdm pwdata
```

1.5 命令列表

PWact 包括如下命令，起始命令为 `pwact`

1.5.1 输出可用命令列表

```
pwact [ -h / --help / help ]
```

您也可以用这条命令检查您的 PWact 是否安装成功 ### 输出 cmd_name 对应的参数列表

```
pwact cmd_name -h
```

1.5.2 初始训练集制备

```
pwact init_bulk param.json resource.json
```

1.5.3 主动学习

```
pwact run param.json resource.json
```

对于上述两个命令，json 文件名称可以用户修改，但是要求 `param.json` 和 `resouce.json` 的输入顺序不能变。

1.5.4 gather_pwdata 命令

搜索主动学习目录下所有探索到的所有结构，并将结果保存到 final_pwdata 目录下，数据格式与主动学习中设置的 'data_format' 格式相同。目录结构如下所示，final_pwdata_list.txt 保存了当前目录下每个 iter 数据的目录。

```
final_pwdata/
final_pwdata_list.txt  iter.0000  iter.0001  iter.0002 ...
```

```
pwact gather_pwdata -i .
```

这里 `-i` 指定主动学习的根目录所在路径（案例中对应 run_iter 目录）。

1.5.5 kill 命令

结束正在执行的 `init_bulk` 任务，如 驰豫（relax）、AIMD 任务

```
# 进入到执行 pwact init_bulk 命令时的所在目录
pwact kill init_bulk
```

结束正在执行的 `run` 任务，包括正在运行的训练、探索（MD）或者标记任务

```
# 进入到执行 pwact run 命令时的所在目录
pwact kill run
```

上面的 kill 命令功能您也可以通过手动操作替代，需要您第一步，结束正在执行的主进程，即执行 pwact init_bulk 或 pwact run 的窗口；第二步，需要您手动结束正在执行的 slurm 任务。

考虑到手动操作可能会误结束您的其他进程，建议您使用命令结束。

使用命令结束进程后，建议您查看命令输出信息，并使用 slurm 命令查看是否有未结束的进程。

1.5.6 filter 命令

测试指定上、下限设置下的选点情况

```
pwact filter -i iter.0000/explore/md -l 0.01 -u 0.02 -s
```

该命令将检测位于 iter.0000/explore/md 目录下（该轮次探索到的所有轨迹）使用下限 0.01 和上限 0.02 的选点结果 (如下面的例子所示)，-s 为可选项，用于指定是否将详细的选点信息做保存。

```
Image select result (lower 0.01 upper 0.02):
 Total structures 972    accurate 20 rate 2.06%    selected 44 rate
 ↪  4.53%    error 908 rate 93.42%

Select by model deviation force:
Accurate configurations: 20, details in file accurate.csv
Candidate configurations: 44
        Select details in file candidate.csv
Error configurations: 908, details in file fail.csv
```

1.6 输入文件

PWact 包括两个输入文件 `param.json` 和 `resource.json`，用于初始数据集制备或者主动学习。PWact 对于两个 JSON 文件中键的大小写输入不敏感。

1.6.1 输入文件 param.json

初始训练集制备 init_param.josn

对构型（VASP、PWmat 格式）进行驰豫、扩胞、缩放、微扰和 AIMD（PWMAT、VASP、CP2K）设置。

主动学习 run_param.josn

主动学习流程中的训练设置（网络结构、优化器）、探索设置（lammps 设置、选点策略）以及标记设置 (VASP/PWmat 自洽计算设置)。

1.6.2 输入文件 resource.json

resource.json

计算集群资源设置，包括对训练、分子动力学（MD）、DFT 计算（SCF、Relax、AIMD）使用的计算节点、CPU、GPU 资源设置，以及对应的运行软件（Lammps、VASP、PWMAT、MatPL）。

1.7 主动学习案例

1.7.1 硅的主动学习

1.7.2 硅的主动学习 (bigmodel+direct)

1.7.3 金银合金的主动学习

2 init_bulk param.json

初始训练集制备，包括对构型（VASP、PWmat 等格式）进行驰豫、阔胞、缩放、微扰和 `AIMD`（支持 DFTB、PWMAT、VASP）设置。参数列表如下。

2.1 data_format

用于设置 init_bulk 执行结束后的得到的数据格式，默认为扩展的 xyz 格式 `extxyz`。

2.2 reserve_work

是否保留临时工作目录，默认值为 `false`，不保存。

2.3 interval

用于设置提取数据时，从轨迹中选取结构的间隔，即在轨迹中，每隔多少个结构选取一个构型，默认值为 `1`。

2.4 sys_config_prefix

用于设置初始构型的路径前缀，可选参数，与 `sys_configs/config` 配合设置。可以是绝对路径或者相对路径，相对路径为当前目录。

例子:`"sys_config_prefix":"/data/structure","config":"atom.config"`，则 `atom.config` 的实际路径是 `/data/structure/atom.config`

2.5 sys_configs

设置构型的文件路径、驰豫（relax）、阔胞（super cell）、缩放晶格（scale）、微扰原子位置（pertub）、AIMD。完整的参数如下例所示。

2.5.1 config

设置构型的文件路径，如果设置了 `sys_config_prefix` 则进行路径拼接，否则使用 config 中设置的路径作为 config 路径。

2.5.2 format

设置构型的文件类型，支持 VASP 的 POSCAR 或者 PWMAT 的 `atom.config` 格式。如果是 POSCAR 文件，则值为 `"vasp/poscar"`，默认值为 `"pwmat/config"`。

2.5.3 relax

是否对 config 做驰豫，默认值为 `"true"`。

2.5.4 relax_input_idx

设置驰豫使用的控制文件，与 relax_input 配合使用，指定控制文件的位置，如 例子 中所示，使用 `relax_input` 中设置的 `relax_etot1.input` 文件作为 PWMAT 控制文件。默认值为 0，即使用 `relax_input` 中的第一个文件作为控制文件。

2.5.5 super_cell

用于阔胞设置，可选参数，如不设置，则对结构不做阔胞。数据格式为 list，支持如下格式输入：`[1, 1, 2]` 或 `[[1,0,0],[0, 2, 0],[0,0,1]]` 或 `[1,0,0,0, 2, 0,0,0,1]`。

2.5.6 scale

用于对晶格的缩放设置，可选参数，如不设置，则对结构不做缩放。数据格式为 list，如 `[0.9, 0.95, 0.96, 0.97]`，表示对结构晶格分别进行 0.9, 0.95，0.96, 0.97 微扰，将得到四个微扰后的结构。

2.5.7 perturb

对结构的原子位置做扰动，配合 `cell_pert_fraction`、`atom_pert_distance` 使用，`perturb` 值为扰动后生成的结构数量。可选参数，不设置则不做扰动。

2.5.8 cell_pert_fraction

对晶格的扰动。对 9 个晶格值分别加上从 [-cell_pert_fraction，cell_pert_fraction] 范围内的均匀分布中随机采样的值，默认值为 `0.03`。

2.5.9 atom_pert_distance

原子坐标的扰动（Angstrom）。对每个原子的三个坐标值，分别加上从 [-atom_pert_distance，atom_pert_distance] 范围内的均匀分布中随机采样的值。默认值为 `0.01`。

2.5.10 aimd

是否对结构做分子动力学模拟，默认值为 true。

2.5.11 aimd_input_idx

设置 AIMD 使用的控制文件，与 aimd_input 配合使用，指定控制文件的位置，如 例子 中所示，使用 `aimd_input` 中设置的 `aimd_etot.input` 文件作为 PWMAT 控制文件。默认值为 0，即 使用 `aimd_input` 中的第一个文件作为控制文件。

2.5.12 bigmodel

是否使用大模型运行 MD，默认值为 False

2.5.13 bigmodel_input_idx

设置 bigmodel 使用的脚本，与 bigmodel_input 配合使用，指定脚本文件的位置。

2.5.14 direct

设置 是否使用 direct 方法筛选结构，默认值 False，与 direct_input 配合使用，direct 为 True 时，必须在 direct_input 中指定 direct 的 脚本文件所在位置。

2.6 sys_config 设置例子

```
"sys_config_prefix": "../../si_example/init_bulk",
"sys_configs": [{"config":"./structures/49.config",
                "relax":true,
                "_relax_input_idx":0,
                "super_cell":[1, 1, 2],
                "scale":[0.9,0.95],
                "perturb":3,
                "cell_pert_fraction":0.03,
                "atom_pert_distance":0.01,
                "aimd":true,
                "_aimd_input_idx":0
                },
                {"config":"./structures/44_POSCAR",
                "format":"vasp/poscar",
                "relax":false,
                "super_cell":[[1,0,0],[0, 2, 0],[0,0,1]],
                "perturb":2,
                "aimd":true,
                "aimd_input_idx": 1
                }
        ]
```

这里设置了 `49.config` 和 44_POSCAR 两个结构，分别是 pwmat/config （默认格式）和 vasp/poscar 格式。- 对 49.config 操作如下：step1. 对 49.config 做 relax，使用的 relax 控制文件为 `relax_input` 中的第一个文件；step2. 对 relax 得到的结构，分别对晶格做 0.9, 0.95 缩放，缩放方式参考；step3. 对缩放后得到两个文件做晶格和原子位置微扰，各自微扰出 3 个结构，微扰方式参考；step4. 对微扰后得到的 6 个结构做 AIMD，使用的 md 控制文件为 `aimd_input` 中设置的第 1 个文件。init_bulk 执行结束后将得到 6 条 AIMD 轨迹。- 对 44_POSCAR 操作如下：step1. 对 49.config 做阔胞，按照 [1,0,0],[0, 2, 0],[0,0,1] 阔胞，阔胞方式参考；step2. 对阔胞后的结构做微扰；step3. 对微扰后得到的 2 个结构做 AIMD，使用的 md 控制文件为 `aimd_input` 中设置的第 2 个文件。init_bulk 执行结束后将得到 2 条 AIMD 轨迹。

因此 init_bulk 执行结束后将得到 6 条轨迹，之后会自动将轨迹提取为 `data_format` 中指定的文件格式。

2.7 dft_style

设置 `Relax` 和 `AIMD` 使用哪种 DFT 计算软件，默认值为 `pwmat`, 也支持 VASP 格式，如果是 VASP 格式，则设置为 `vasp`。

2.8 pseudo

设置 PWMAT 或 VASP 赝势文件所在路径，为 list 格式，赝势文件路径可以为绝对路径或相对路径（相对于当前路径)。

2.9 gaussian_param

CP2K 或 PWMAT 高斯基组参数设置，`basis_set_file` 和 `potential_file` 指定基组和势函数文件路径。`atom_list`, `basis_set_list`, `potential_list` 配合使用，分别指定元素对应的基组和势函数设置。kspacing 用于设置 K 点，用法与 PWMAT KSPACKING 设置 相同。

```
"gaussian_param": {
    "basis_set_file":"./init_bulk/BASIS_MOLOPT_1",
    "potential_file":"./init_bulk/POTENTIAL_1",
    "atom_list":["Si"],
    "kspacing" :0.4,
    "basis_set_list":["SZV-MOLOPT-SR-GTH"],
    "potential_list":["GTH-PBE-q4"]
}
```

2.10 relax_input

设置 Relax 的 输入控制文件。如果存在多个 relax 控制文件，则按照 list 格式组织。详细的设置请参考下面的例子。

2.11 aimd_input

设置 AIMD 的 输入控制文件。如果存在多个 aimd 控制文件，则按照 list 格式组织，对于只是用单个文件的情况，也可以设置为 dict 格式。

2.11.1 input

设置输入控制文件的路径，可以为绝对路径或相对路径（相对于当前路径）。

2.11.2 kspacing

该参数为 PWMAT 的输入参数，用于设置 K 点，可选参数。如果在 etot.input 文件中未设置 `MP_N123` 参数，则使用该参数设置 K 点。如果文件中已经设置了 `MP_N123`，则对该参数的设置会造成错误，请确保 `MP_N123` 与 `kspacing` 只能同时存在一个。该参数为 PWMAT 的输入参数，用于设置 K 点，可选参数。如果在 etot.input 文件中未设置 `MP_N123` 参数，则使用该参数设置 K 点。

如果 `etot.input` 文件中未设置 `MP_N123`，且 `kspacing` 未设置，则采用默认设置 `kspacing` 值为 `0.5`。

注意，不能同时设置 `MP_N123` 与 `kspacing`。

2.11.3 flag_symm

该参数为 PWMAT 的输入参数，用于设置 K 点，可选参数。对于 Relax 或者 SCF 计算，默认值为 `0`, 对于 AIMD 计算，默认值为 3。

2.12 bigmodel_input

设置 大模型的脚本文件。如果存在多个脚本文件，则按照 list 格式组织。

对大模型的接口设置，请参考例子 si_direct_bigmodel。

2.13 direct_input

设置 direct 采样的脚本文件，为单文件路径。

对 direct 的接口设置，请参考例子 si_direct_bigmodel。

2.14 完整例子

```
{
    "reserve_work": true,
    "sys_configs":[{
        "config":"atom.config",
        "relax":true,
        "relax_input_idx":1,
        "super_cell":[1, 1, 2],
```

```
            "scale":[0.9,0.95],
            "perturb":20,
            "cell_pert_fraction":0.03,
            "atom_pert_distance":0.01,
            "aimd":true,
            "aimd_input_idx":0
        }],

    "dft_style":"PWmat",
    "pseudo" : ["/path/Si.SG15.PBE.UPF"],

    "relax_input":[{
            "input":"relax_etot.input",
            "kspacing":0.5,
            "flag_symm":"0"
        },{
            "input":"relax_etot1.input",
            "kspacing":0.3,
            "flag_symm":"0"
        },{
            "input":"relax_etot2.input",
            "kspacing":0.4,
            "flag_symm":"0",
            "_flag":"1 个整数, relax or scf 0 , aimd 3, 磁性体系 2"
        }],

    "aimd_input":[{
            "input":"aimd_etot.input",
            "kspacing":0.5,
            "flag_symm":"3"
        },{
            "input":"aimd_etot1.input",
            "kspacing":0.5,
            "flag_symm":"3"
        }]
    }
```

2.14.1 上述参数作用

1. 对 atom.config 做 relax;
2. 对 relax 后的结构做阔胞（1,1,2）;
3. 对阔胞后的结构对晶格分别进行 0.9、0.95 缩放;
4. 对缩放后得到两个结构的原子位置微扰，各自扰动生成 20 个结构;

5. 对扰动后得到的 40 个结构做 AIMD 模拟；
6. 将 AIMD 轨迹自动提取为 PWdata 数据格式作为预训练数据。

2.14.2 relax_input 和 aimd_input

kspacing和flag_symm是 PWMAT 的 K 点设置，如果没有在 etot.input 文件中设置 MP_N123 参数，那么程序会使用这两个参数设置 K 点。因此，如果您已经 etot.input 设置了 MP_N123 ，那么，您可以将relax_input 和 aimd_input 简写为如下形式

```
"relax_input":[
    "relax_etot.input",
    "relax_etot1.input",
    "relax_etot2.input"
],

"aimd_input":[
    "aimd_etot.input"
    "aimd_etot1.input"
]
```

如果您对在 sys_configs 中的所有结构，都使用同一个 relax 或者 aimd，那么您可以进一步将参数简写为

```
"relax_input":"relax_etot.input",
"aimd_input": "aimd_etot.input"
```

此时，在 sys_configs 您不必再写 relax_input_idx 和 aimd_input_idx 参数，此时您的 sys_configs可以写为如下所示。

```
"sys_configs":[{
    "config":"atom.config",
    "relax":true,
    "super_cell":[1, 1, 2],
    "scale":[0.9,0.95],
    "perturb":20,
    "cell_pert_fraction":0.03,
    "atom_pert_distance":0.01,
    "aimd":true
}]
```

2.14.3 pwmat 设置例子

pwmat K 点通过在控制文件中设置 MP_N123，例子：

```
    "dft_style":"PWmat",
    "relax_input":["relax-agau-etot.input", "relax_ag-etot.input",
     ↪  "relax_au-etot.input"],
    "aimd_input": ["aimd_agau-etot.input",   "aimd_ag-etot.input",
     ↪  "aimd_au-etot.input" ],
    "pseudo" : ["../Ag.SG15.PBE.UPF", "../Au.SG15.PBE.UPF"]
```

pwmat K 点通过 kspacing 设置，例子：

```
    "dft_style":"PWmat",
    "relax_input":[{
            "input":"relax_etot0.input",
            "kspacing":0.5,
            "flag_symm":"0"
        },{
            "input":"relax_etot1.input",
            "kspacing":0.3,
            "flag_symm":"0"
        },{
            "input":"relax_etot2.input",
            "kspacing":0.4,
            "flag_symm":"0",
            "_flag":"1 个整数, relax or scf 0 , aimd 3, 磁性体系 2"
        }],

    "aimd_input":[{
            "input":"aimd_etot0.input",
            "kspacing":0.5,
            "flag_symm":"3"
        },{
            "input":"aimd_etot1.input",
            "kspacing":0.5,
            "flag_symm":"3"
        }],
    "pseudo" : ["../Ag.SG15.PBE.UPF", "../Au.SG15.PBE.UPF"]
```

pwmat gaussian 基组设置例子：

```
    "dft_style":"PWmat",
    "relax_input":["relax_etot.input",
     ↪  "relax_etot1.input","relax_etot2.input"],
    "aimd_input": ["aimd_etot1.input", "aimd_etot2.input"],
    "gaussian_param": {
        "basis_set_file":"./BASIS_MOLOPT_1",
        "potential_file":"./POTENTIAL_1",
```

```
        "atom_list":["Si"],
        "basis_set_list":["SZV-MOLOPT-SR-GTH"],
        "potential_list":["GTH-PBE-q4"]
    }
```

2.14.4 vasp 设置例子

```
    "dft_style":"vasp",
    "relax_input":["INCAR_relax_AgAu", "INCAR_relax_Ag",
     ↪  "INCAR_relax_Au"],
    "aimd_input": ["INCAR_md_AgAu",   "INCAR_md_Ag", "INCAR_md_Au" ],
    "pseudo" : ["../Ag_POTCAR", "../Au_POTCAR"]
```

2.14.5 cp2k 设置例子

```
    "dft_style":"cp2k",
    "gaussian_param": {
        "basis_set_file":"../BASIS_MOLOPT_1",
        "potential_file":"../POTENTIAL_1",
        "atom_list":["Ag", "Au"],
        "basis_set_list":["SZV-MOLOPT-SR-GTH-q11",
         ↪  "SZV-MOLOPT-SR-GTH-q11"],
        "potential_list":["GTH-PBE", "GTH-PBE"],
        "kspacing":0.5
    },
    "aimd_input":["aimd_cp2k_AgAu.inp", "aimd_cp2k_Ag.inp",
     ↪  "aimd_cp2k_Au.inp"],
    "relax_input":["relax_cp2k_AgAu.inp", "relax_cp2k_Ag.inp",
     ↪  "relax_cp2k_Au.inp"]
```

3 run_iter param.json

4 param.json

训练设置（网络结构、优化器）、探索设置（lammps 设置、选点策略）以及标记设置（VASP/PWmat 自洽计算设置）。参数列表如下所示

4.1 reserve_work

是否保留临时工作目录，默认值为 `false`，每轮次主动学习执行结束之后，自动删除临时工作目录。

4.2 reserve_md_traj

是否保留 md 运行轨迹，默认值为 `false`，每轮次主动学习执行结束之后，自动删除 md 运行轨迹文件。

4.3 reserve_scf_files

是否保留自洽计算的所有结果文件，默认值为 `false`，设置为 `false` 之后，每轮次主动学习结束之后，对于 PWMAT 自洽计算，只保留 `REPORT`, `etot.input`,`OUT.MLMD`, `atom.config` 四个文件，对于 VASP 只保留 `OUTCAR`, `POSCAR`, `INCAR` 三个文件。

4.4 data_format

用于设置主动学习中初始训练集、采集到的数据集格式，默认为扩展的 xyz 格式 `extxyz`。

4.5 init_data

初始训练集所在目录，为 list 格式。可以是绝对路径或者相对路径（当前目录）。

4.6 valid_data

验证集所在目录，为 list 格式，可以是绝对路径或者相对路径（当前目录）。如不设置，则在主动学习训练模型过程中不输出验证集结果。

4.7 init_model_list

用于设置初始探索模型，如果已有 MatPL 训练的 DP 或者 NEP 力场，并且希望从这些力场开始探索工作，则将力场文件路径写入 init_model_list 即可。

注意，这里要求力场数量要与 strategy/model_num 中一致，模型类型要与 `train/model_type` 中的模型类型一致。并且模型的训练参数将会自动从力场文件中提取，在 `train_input_file` 或者 `train` 字典中设置的模型网络和描述符参数将失效。

4.8 use_pre_model

在当前步的探索中，使用上一步训练得到的力场，默认值为 true。

4.9 train

模型训练参数，用于指定模型网络结构、优化器。详细的参数设置参考 `MatPL` 训练参数。您可以像如下例子中所示，设置训练的全部参数，也可以使用单独的 json 文件，只需要在参数 `train_input_file` 中指定训练的 json 文件所在路径即可。

4.9.1 train_input_file

可选参数，如果您有单独的 MatPL 输入文件，您可以使用该参数指定文件所在路径。否则您需要设置如下例中所示参数。参数的详细解释您在可以在 MatPL 参数列表中查看。

```
"train": {
    "model_type": "DP",
    "atom_type": [
        14
    ],
    "seed": 2023,
    "model": {
        "descriptor": {
            "Rmax": 6.0,
            "Rmin": 0.5,
            "M2": 16,
            "network_size": [25, 25, 25]
        },
        "fitting_net": {
            "network_size": [50, 50, 50, 1]
        }
    },
    "optimizer": {
        "optimizer": "LKF",
        "epochs": 30,
        "batch_size": 4,
        "print_freq": 10,
        "block_size": 5120,
        "kalman_lambda": 0.98,
        "kalman_nue": 0.9987,
        "train_energy": true,
        "train_force": true,
        "train_virial": false,
        "pre_fac_force": 2.0,
        "pre_fac_etot": 1.0,
        "pre_fac_virial": 1.0
    }
```

```
    }
```

由于 MatPL 中设置的默认参数已经能够支持大部分的训练需求，因此，您可以简写为如下形式，将采用标准的 DP 模型 使用 LKF 优化器训练。

```
"train": {
      "model_type": "DP",
      "atom_type": [14]
}
```

PWact 同时支持 MatPL 的 DP 和 NEP 力场。

4.10 strategy

用于设置主动学习的不确定性度量方法，以及是否采用模型压缩做加速。

4.10.1 uncertainty

用于设置不确定性度量策略，当前支持多模型委员会查询方法 (`committee`) 。

4.10.2 lmps_tolerance

当 lammps 部分轨迹由于各种原因（如力场不准确导致丢失了原子、原子距离太近等）造成 MD 过程为正常执行结束时，是否终止主动学习过程。默认值为 true，即不终止。

4.10.3 lower_model_deiv_f

该参数用于设置偏差的下界，如果偏差值小于该下界，则认为模型对构型的预测准确，不需要标注，默认值为 `0.05`。

4.10.4 upper_model_deiv_f

该参数用于设置偏差的上界，如果偏差值大于该上界，则该构型本身不符合物理意义，不需要标注，默认值为 `0.15`。

这里使用的最大偏差值，计算公式如下所示：

$$\varepsilon_t = \max_i \left(\sqrt{\frac{\sum_1^w \left\| F_{w,i}(R_t) - \hat{F}_i \right\|^2}{W}} \right), \quad \hat{F}_i = \frac{\sum_1^W F_{w,i}}{W}$$

这里 W 为模型数量，i 为原子下标。

4.10.5 model_num

该参数用于设置使用 `committee` 方法作为不确定性度量时使用的模型数量，默认值为 `4`，该值的设置应该 >=3。

4.10.6 max_select

该参数用于设置一轮次主动学习中，对于未设置 `select_sys` 参数的每个初始探索结构对应最大选取构型用于标注的数量。当待标注结构数目超过该值时，将随机从待标注结构中选择 `max_select` 个结构做标注。默认不设置，即不做限制。

例如对于如下 md 探索设置，由于未设置 `select_sys`，如果设置了 `max_select`，则对于 `sys_idx` 中指定的这两个结构，分别最多采集 `max_select` 个结构，因此这里对于该 md 探索设置，最多采集 $2 ×max_select $ 个结构用于标记。

4.10.7 direct

该参数用于设置是否开启 direct 采样，用于对多模型偏差选出的结构进一步过滤，去除相似的结构。默认值为 False。

4.10.8 direct_script

该参数用于设置 direct 方法使用的处理脚本，direct 为 True 时，必须指定处理脚本。

对 direct 方法的接口设置，请参考例子 si_direct_bigmodel。

4.10.9 compress

该参数用于设置是否对模型做压缩，经过压缩后的模型精度会略有下降，但是模拟速度会有翻倍提升。默认值为 `false`，即不使用模型压缩。

4.10.10 compress_order

该参数用于设置模型压缩的方式，默认值为 `"compress_order":3`，即使用三阶多项式压缩。也可以设置为 `"compress_order":5`，即使用五阶多项式压缩，相比于三阶多项式精度会更高，但是速度比三阶稍慢。

4.10.11 Compress_dx

该参数用于设置模型压缩是的网格大小，默认值为 `"Compress_dx":0.01`。

4.10.12 例子

对于 committee 方式的选点策略

```
"strategy": {
    "uncertainty":"committee",
    "lower_model_deiv_f": 0.1,
    "upper_model_deiv_f": 0.2,
    "model_num": 4,
    "max_select": 50,
    "compress": false
}
```

如果您需要开启模型压缩，则

```
"strategy": {
    "uncertainty":"committee",
    "lower_model_deiv_f": 0.1,
    "upper_model_deiv_f": 0.2,
    "model_num": 4,
    "max_select": 50,
    "compress": true,
    "compress_order":3,
    "Compress_dx":0.01
}
```

5

5.1 explore

用于设置主动学习每个轮次探索过程的分子动力学设置

5.1.1 sys_config_prefix

用于设置待探索的结构文件路径前缀，可选参数，与 `sys_configs` 配合使用。可以是绝对路径或者相对路径，相对路径为当前目录。

5.1.2 sys_configs

设置待探索的结构文件路径，如果设置了 `sys_config_prefix` 则进行路径拼接，否则使用 sys_configs 中设置的路径作为 config 路径。

该参数为 list 格式，对于 PWMAT 格式结构文件，直接写出文件路径即可，对于 VASP 格式结构文件，需要设置 `format` 格式，如下例中所示。

```
"sys_config_prefix": "../../structures",
"sys_configs": [
                {"config":"POSCAR", "format":"vasp/poscar"},
                "49.config",
                "45.config",
                "41.config",
                "37.config",
                "33.config",
                "29.config",
                "25.config",
                "21.config",
                "17.config",
                "1.config"
    ]
```

例如对于 `49.config` 文件，它的文件路径为`../../structures/49.config`。

5.1.3 lmps_prefix

可选参数，用于设置 lammps 探索的输入控制文件路径前缀，可选参数，与 `lmps_in` 配合使用。可以是绝对路径或者相对路径，相对路径为当前目录。

例子: `"lmps_prefix":"/data/in_lmps_files"`, `"lmps_in":"in0.lmps"`，则 `in0.lmps` 的实际路径是 `/data/in_lmps_files/in0.lmps`。

5.1.4 lmps_in

可选参数，设置设置 lammps 探索的输入控制文件路径，如果设置了 `lmps_prefix` 则进行路径拼接，否则使用 lmps_in 中设置的路径作为 lammps 控制文件的路径。

该参数为 list 格式，如下例中所示。

```
"lmps_prefix": "../../in_lmps_files",
"lmps_in": [
                "in0.lmps",
                "in1.lmps",
                "in2.lmps",
                "in3.lmps",
                "in4.lmps",
                "in5.lmps"
    ]
```

5.1.5 md_jobs

设置每个轮次的主动学习分子动力学参数。为 list 格式，第 `i` 个元组代表第 `i` 个轮次主动学习对应设置。每个元组内部可以为 dict 格式，或者 dict 格式的 list 数组。注意：以下 md 设置中，对于时间的单位都使用 `units metal`。

对于 lammps 的输入控制，pwact 提供了两种方式。第一种是在 param.json 中提供的关键字设置，控制探索需要的步数、以及 lammps 温度、压强、系综，可参考例子。第二种是通过用户提供的 `lammps.in` 文件控制，参数为 `lmps_prefix` 和 `lmps_in`。可参考 金银合金主动学习操作案例。

`lmps_in` 设置方式在 `>= pwact-0.4` 版本中开始支持。

对于 `lmps_in`，如果用户提供了 lammps.in 文件，pwact 在运行时，会自动维护 lammps.in 文件中的以下字段。

- “dump_freq”，该参数通过 trj_freq 设置，用于设置每隔多少步采样一次
- “units”、“boundary”、“atom_style”，由于 MatPL 力场只支持周期性的体系模拟，因此这三个关键字是固定格式，内容为

  ```
  units           metal
  boundary        p p p
  atom_style      atomic
  ```

- “restart”，pwact 中自动设置每个 1 万步保存一次运行状态
- “read_data”，该参数用设置初始结构所在路径，在 pwact 在探索时该值会自动设置为所需路径
- “mass”、“pair_style”、“pair_coeff”，该三个参数用于设置机器学习的力场，以对硅元素体系的探索为例，在 pwact 在探索时该值会自动设置为：

  ```
  mass    1    28.086
  pair_style   matpl  0_torch_script_module.pt
  ↪  1_torch_script_module.pt 2_torch_script_module.pt
  ↪  3_torch_script_module.pt  out_freq ${DUMP_FREQ}  out_file
  ↪  model_devi.out
  pair_coeff   * * 14
  ```

- “dump”，该值用于设置轨迹的保存格式，在 pwact 中该值会自动设置为如下内容，并插入到 lammps.in 文件中的第一个 run 命令所在行前面，这里 DUMP_FREQ 值为trj_freq参数中所设值：

  ```
  dump 1 all custom ${DUMP_FREQ} traj/*.lammpstrj id type x y z
  ↪  fx fy fz
  ```

- “dump_modify”，如果用户设置了该值，pwact 在使用时会自动将该行的‘dump_modify dump-ID’替换为‘dump_modify 1’，即设置 dump-ID 与 dump 中一致，其他不变

这里以如下设置为例，该 lmp.in 文件为硅的 lammps 模拟输入文件：

```
variable        NSTEPS          equal 400
variable        THERMO_FREQ     equal 5
variable        DUMP_FREQ       equal 5
variable        restart         equal 0
variable        TEMP            equal 500.000000
variable        PRESS           equal 100.000000
variable        TAU_T           equal 0.100000
variable        TAU_P           equal 0.500000

units           metal
boundary        p p p
atom_style      atomic

neighbor        1.0 bin
neigh_modify    delay 10

box              tilt large
if "${restart} > 0" then "read_restart lmps.restart.*" else
 ↪  "read_data lmp.config"
change_box       all triclinic

thermo_style    custom step temp pe ke etotal press vol lx ly lz xy xz
 ↪  yz
thermo          ${THERMO_FREQ}
restart         10000 lmps.restart

if "${restart} == 0" then "velocity        all create ${TEMP} 76752"
fix             1 all npt temp ${TEMP} ${TEMP} ${TAU_T} iso ${PRESS}
 ↪  ${PRESS} ${TAU_P}

timestep        0.001000
run             ${NSTEPS} upto
```

运行时，lmp.in 会替换为如下内容：

```
variable        DUMP_FREQ       equal 5
variable        restart         equal 0
units           metal
boundary        p p p
atom_style      atomic

if "${restart} > 0" then "read_restart lmps.restart.*" else
 ↪  "read_data lmp.config"

mass   1    28.086
```

```
pair_style   matpl  0_torch_script_module.pt 1_torch_script_module.pt
 ↪  2_torch_script_module.pt 3_torch_script_module.pt  out_freq
 ↪  ${DUMP_FREQ} out_file model_devi.out
pair_coeff       * * 14

variable        NSTEPS          equal 400
variable        THERMO_FREQ     equal 5
variable        TEMP            equal 500.000000
variable        PRESS           equal 100.000000
variable        TAU_T           equal 0.100000
variable        TAU_P           equal 0.500000

neighbor        1.0 bin
neigh_modify    delay 10

box              tilt large
change_box       all triclinic


thermo_style    custom step temp pe ke etotal press vol lx ly lz xy xz
 ↪  yz
thermo          ${THERMO_FREQ}

fix             1 all npt temp ${TEMP} ${TEMP} ${TAU_T} iso ${PRESS}
 ↪  ${PRESS} ${TAU_P}

timestep        0.001000
dump            1 all custom ${DUMP_FREQ} traj/*.lammpstrj id type x y
 ↪  z fx fy fz
restart         10000 lmps.restart

if "${restart} == 0" then "velocity        all create ${TEMP} 75740"
run             ${NSTEPS} upto
```

5.1.5.1 sys_idx 用于设置 md 的初始结构，为 list 格式，值为 `sys_configs`中的结构下标，这里可以指定多个结构。

5.1.5.2 select_sys 与 `sys_idx` 配合使用，用于限制 `sys_idx` 中每个初始探索结构的最多用于标注的构型数量，默认不设置，采用 `max_select`中的设置。如果参数 `max_select` 也未设置，将采用默认值 100。例如对于如下设置：

```
"sys_idx": [0, 1],
"select_sys":[20, 30],
```

sys_idx 指定了 `0` 号结构 POSCAR 和 1 号结构 49.config，在 `0` 号结构对应的轨迹中最多选取 `20` 个结构用于标注，在 `1` 号结构对应的轨迹中最多最多选取 `30` 个结构用于标注。如果不设置 select_sys，则对 0 和 1 号结构分别最多选取`max_select`个结构做标记。

您也可以设置为`"select_sys":20`，等效于`"select_sys":[20, 20]`。

5.1.5.3 trj_freq

用于设置轨迹采样频率（`thermo`），默认值为 `10`，即间隔 10 步采样一次。

5.1.5.4 lmps_in_idx

配合 `lmps_in` 使用。设置对`sys_idx`中对应的初始结构做分子动力学探索的 lammps.in 文件所在路径。使用如下例子中所示：

```
"md_jobs": [
    [{
        "sys_idx": [ 1,3,4],
        "select_sys":[10,15,20],
        "lmps_in_idx":[0, 1, 2],
        "trj_freq": 5
    },{
        "sys_idx": [0, 1],
        "lmps_in_idx":3
    }
    ]]
```

在该例中，对于" sys_idx" : [1,3,4] 指定了需要探索的结构，即上例中的 49.config、41.config、37.config。分别设置 lammps 控制文件为 in0.lmps、in1.lmps、in2.lmps。设置每隔 5 步，采样一次。

对于" sys_idx" : [0, 1] 指定的结构 49.config、41.config，设置 lammps 输入控制文件 in3.lmps。采样间隔用默认值，每个 10 步采样一次。

存在 lmps_in_idx 设置时，`ensemble`、`nsteps`、`md_dt`、`press`、`taup`、`temps`、`taut` 参数将自动失效。

5.1.5.5 ensemble

用于第一种 lammps 输入控制方式。设置 系综，默认值 `"nve"`，支持如下设置：

`npt`、`npt-i` 或 `npt-iso` 对应 lammps 设置

```
fix  1 all npt temp ${TEMP} ${TEMP} ${TAU_T} iso ${PRESS} ${PRESS}
↪  ${TAU_P}
```

`npt-a` 或 `npt-aniso` 对应 lammps 设置

```
fix  1 all npt temp ${TEMP} ${TEMP} ${TAU_T} aniso ${PRESS} ${PRESS}
↪  ${TAU_P}
```

`npt-t`、`npt-tri` 对应 lammps 设置

```
fix  1 all npt temp ${TEMP} ${TEMP} ${TAU_T} tri ${PRESS} ${PRESS}
 ↪  ${TAU_P}
```

nvt 对应 lammps 设置

```
fix  1 all nvt temp ${TEMP} ${TEMP} ${TAU_T}
```

nve 对应 lammps 设置

```
fix  1 all nve
```

5.1.5.6 nsteps 用于第一种 lammps 输入控制方式。设置 md 总的步数，为必选参数，需要用户提供。#### md_dt

用于第一种 lammps 输入控制方式。用于设置 `timestep`，默认值为 `0.001`，即 1 飞秒。

5.1.5.7 press 用于第一种 lammps 输入控制方式。用于设置 md 探索的压强，为 list 格式。

5.1.5.8 taup 用于第一种 lammps 输入控制方式。稳压器的耦合时间（ps），默认值 `0.5`。

5.1.5.9 temps 用于第一种 lammps 输入控制方式。用于设置 md 探索的温度，为 list 格式。

5.1.5.10 taut 用于第一种 lammps 输入控制方式。恒温器的耦合时间（ps），默认值 `0.1`。

5.1.5.11 boundary 用于第一种 lammps 输入控制方式。设置模拟系统的边界条件，由于 MatPL 力场只支持对周期性体系的模拟，因此该值为 `true`，即采用 p p p。不需要用户设置。

5.1.5.12 例子

```
"explore": {
    "sys_config_prefix": "./init_bulk/collection/init_config_0",
    "sys_configs": [
        {"config":"0.95_scale.poscar", "format":"vasp/poscar"},
        {"config":"0_pertub.poscar", "format":"vasp/poscar"},
        {"config":"0_pertub.poscar", "format":"vasp/poscar"}
    ],
    "md_jobs": [
        [{
            "ensemble": "npt",
            "nsteps": 1000,
            "md_dt": 0.002,
            "trj_freq": 10,
```

```
            "sys_idx": [0, 1],
            "temps": [500, 800],
            "taut":0.1,
            "press" :[ 100,200],
            "taup": 0.5,
            "boundary":true
            },{
            "ensemble": "nvt",
            "nsteps": 1000,
            "md_dt": 0.002,
            "trj_freq": 10,
            "sys_idx": [2],
            "temps": [400],
            "taut":0.1,
            "boundary":true
        }]
    ]
}
```

在上例中，配置了一个轮次的主动学习，执行 "md_jobs" 中两个 dict 中配置的 lammps 模拟。

对于第一个 dict, 使用 npt 系综，在 sys_idx 配置了两个构型，对应 0.95_scale.poscar 和 0_pertub.poscar。温度和压强的列表分别为 [500, 800] 和 [100,200]，意思是对这两个结构分别在温度、压强组合为 [500, 100]、[500, 800]、[800, 100]、[800, 200] 下执行 lammps 模拟，模拟 1000 步，输出频率 10 步，单步时间长度 2 飞秒。模拟结束后，会获得 8 条轨迹。

对于第二个 dict，使用 nvt 系综，在 sys_idx 配置了 1 个构型，对应 0_pertub.poscar，温度为 400，即在 温度 400K 下做 lammps 模拟。模拟结束后获得 1 条轨迹。

下面是" ensemble" : "npt" , "nsteps" : 1000, "md_dt" : 0.002, "trj_freq" : 10, "sys_idx" : 0, "temps" : 500, "taut" :0.1, "press" :100, "taup" : 0.5, "boundary" :true 设置下，自动生成的 lammps.in 输入控制文件内容:

```
variable        NSTEPS          equal 400
variable        THERMO_FREQ     equal 5
variable        DUMP_FREQ       equal 5
variable        restart         equal 0
variable        TEMP            equal 500.000000
variable        PRESS           equal 100.000000
variable        TAU_T           equal 0.100000
variable        TAU_P           equal 0.500000

units           metal
boundary        p p p
atom_style      atomic
```

```
neighbor        1.0 bin
neigh_modify    delay 10

box              tilt large
if "${restart} > 0" then "read_restart lmps.restart.*" else
 ↪  "read_data lmp.config"
change_box       all triclinic

mass   1    28.086
pair_style   matpl  0_torch_script_module.pt 1_torch_script_module.pt
 ↪  2_torch_script_module.pt 3_torch_script_module.pt  out_freq
 ↪  ${DUMP_FREQ} out_file model_devi.out
pair_coeff       * * 14

thermo_style    custom step temp pe ke etotal press vol lx ly lz xy xz
 ↪  yz
thermo          ${THERMO_FREQ}
dump            1 all custom ${DUMP_FREQ} traj/*.lammpstrj id type x y
 ↪  z fx fy fz
restart         10000 lmps.restart

if "${restart} == 0" then "velocity        all create ${TEMP} 76752"
fix             1 all npt temp ${TEMP} ${TEMP} ${TAU_T} iso ${PRESS}
 ↪  ${PRESS} ${TAU_P}

timestep        0.001000
run             ${NSTEPS} upto
```

文件中 velocity 中的 `76752` 为随机生成值。## DFT

设置自洽计算，为 dict 格式。

5.1.6 dft_style

设置标注（自洽计算）使用哪种 DFT 计算软件，默认值为 pwmat, 也支持 VASP 或 CP2K 格式，分别设置为 `vasp` 或 `cp2k`。如果设置为 `bigmodel`，则必须在 `bigmodel_script` 中设置处理脚本。

5.1.7 bigmodel_script

用于设置 大模型做标记（推理能量和受力）时的处理脚本。

对大模型标记的接口设置，请参考例子 si_direct_bigmodel。

5.1.8 input

设置输入控制文件的路径，可以为绝对路径或相对路径（相对于当前路径）。

5.1.9 kspacing

该参数为 PWMAT 的输入参数，用于设置 K 点，可选参数。如果在 etot.input 文件中未设置 `MP_N123` 参数，则使用该参数设置 K 点。不能同时设置 `MP_N123` 与 `kspacing`。

如果 `etot.input` 文件中未设置 `MP_N123`，且 `kspacing` 未设置，则采用默认设置 `kspacing` 值为 `0.5`。

5.1.10 flag_symm

该参数为 PWMAT 的输入参数，用于设置 K 点，可选参数。对于 Relax 或者 SCF 计算，默认值为 0, 对于 AIMD 计算，默认值为 3。

5.1.11 pseudo

设置 PWMAT 或 VASP 赝势文件所在路径，为 list 格式，赝势文件路径可以为绝对路径或相对路径（相对于当前路径）。

5.1.12 gaussian_param

CP2K 或 PWMAT 高斯基组参数设置，`basis_set_file` 和 `potential_file` 指定基组和势函数文件路径。`atom_list`, `basis_set_list`, `potential_list` 配合使用，分别指定元素对应的基组和势函数设置。kspacing 用于设置 K 点，用法与 PWMAT KSPACKING 设置 相同。

```
"gaussian_param": {
    "basis_set_file":"./init_bulk/BASIS_MOLOPT_1",
    "potential_file":"./init_bulk/POTENTIAL_1",
    "atom_list":["Si"],
    "kspacing" :0.4,
    "basis_set_list":["SZV-MOLOPT-SR-GTH"],
    "potential_list":["GTH-PBE-q4"]
}
```

5.1.13 DFT 设置例子

由于自洽计算任务使用的输入控制相同，因此只需要单文件的设置，对于不同的 DFT 软件，分别设置如下。

5.1.13.1 PWMat 设置 对于 PWMAT，设置与 INIT_BULK 中相似，如果您未在 scf_etot.input 中指定“MP_N123”参数，则您需要设置 kspacing 和 flag_symm 参数。

```
"DFT": {
        "dft_style": "pwmat",
        "input": "scf_etot.input",
        "kspacing":0.5,
        "flag_symm":0,
        "pseudo" : ["../Ag.SG15.PBE.UPF","../Au.SG15.PBE.UPF"],
        "_flag":"1 个整数, or scf 0 , aimd 3, 磁性体系 2"
}
```

5.1.13.2 VASP 设置

```
"DFT": {
        "dft_style":"vasp",
        "input":"INCAR_scf",
        "pseudo" : ["../Ag_POTCAR", "../Au_POTCAR"]
}
```

5.1.13.3 CP2K 设置

```
"DFT": {
        "dft_style":"cp2k",
        "input":"scf.inp",
        "gaussian_param": {
            "basis_set_file":"../BASIS_MOLOPT_1",
            "potential_file":"../POTENTIAL_1",
            "atom_list":["Ag", "Au"],
            "basis_set_list":["SZV-MOLOPT-SR-GTH-q11",
             ↪  "SZV-MOLOPT-SR-GTH-q11"],
            "potential_list":["GTH-PBE", "GTH-PBE"],
            "_kspacing":0.5
        }
}
```

注意，请在 scf_cp2k.inp 文件中设置 IGNORE_CONVERGENCE_FAILURE 参数，设置后对于不收敛的情况，CP2K 将正常退出，pwact 在检测到 CP2K 正常退出后，将会继续向下执行。否则，pwact 在检测到 CP2K 异常停止后，将重新提交任务三次，三次失败后，将会停止运行。

5.1.14 案例参考

如下例子，为一个标准的主动学习流程，两个轮次的主动学习，采用多模型委员会查询策略。更多使用案例，请参考源码根目录的 `example`。

```
{
  "reserve_work": false,
  "reserve_md_traj": false,
  "reserve_scf_files": false,

  "init_data": ["/path/init_data"],

  "train": {
    "_train_input_file": "std_si.json",

    "model_type": "DP",
    "atom_type": [14],
    "seed": 2023,
    "recover_train": true,
    "model": {
      "descriptor": {
        "Rmax": 6.0,
        "Rmin": 0.5,
        "M2": 16,
        "network_size": [25, 25, 25]
      },
      "fitting_net": {
        "network_size": [50, 50, 50, 1]
      }
    },
    "optimizer": {
      "optimizer": "LKF",
      "epochs": 10,
      "batch_size": 16,
      "print_freq": 10,
      "block_size": 5120,
      "kalman_lambda": 0.98,
      "kalman_nue": 0.9987,
      "train_energy": true,
      "train_force": true,
      "train_virial": false,
      "pre_fac_force": 2.0,
      "pre_fac_etot": 1.0,
      "pre_fac_virial": 1.0
    }
  },

  "strategy": {
    "uncertainty": "committee",
```

```
    "lower_model_deiv_f": 0.05,
    "upper_model_deiv_f": 0.15,
    "model_num": 4,
    "max_select": 10
  },

  "explore": {
    "sys_config_prefix": "/path/structures",
    "sys_configs": [
      { "config": "POSCAR", "format": "vasp/poscar" },
      "atom1.config",
      "atom2.config",
      "atom3.config",
      "atom4.config"
    ],
    "md_jobs": [
      [
        {
          "ensemble": "nvt",
          "nsteps": 1000,
          "md_dt": 0.002,
          "trj_freq": 10,
          "taup": 0.5,
          "sys_idx": [0, 1],
          "temps": [500, 700],
          "taut": 0.1,
          "boundary": true
        },
        {
          "ensemble": "npt",
          "nsteps": 1000,
          "md_dt": 0.002,
          "trj_freq": 10,
          "press": [100.0, 200.0],
          "taup": 0.5,
          "sys_idx": [0, 3],
          "temps": [500, 700],
          "taut": 0.1,
          "boundary": true
        }
      ],
      {
        "ensemble": "nvt",
```

```
        "nsteps": 4000,
        "md_dt": 0.002,
        "trj_freq": 10,
        "press": [100.0, 200.0],
        "taup": 0.5,
        "sys_idx": [0, 1],
        "temps": [500, 700],
        "taut": 0.1,
        "boundary": true
      }
    ]
  },
  "DFT": {
    "dft_style": "pwmat",
    "input": "scf_etot.input",
    "kspacing": 0.5,
    "flag_symm": 0,
    "pseudo": ["path/Si.SG15.PBE.UPF"]
  }
}
```

6 resource.json

6.1 resource.json

设置计算集群资源，包括对训练、分子动力学（MD）、DFT 计算（SCF、Relax、AIMD）使用的计算节点、CPU、GPU 资源以及对应的运行软件（Lammps、VASP、PWMAT、MatPL）。

所有可设置参数按照用途分为 `train`, `explore`, DFT, `direct` 四种模块，每个模块中的参数意义相同。

对于初始训练集制备（init_bulk）可设置 `explore`, DFT, `direct`，这里以 mcloud 环境设置为例。explore 用于设置大模型 MD 环境，direct 用于设置 direct 采样环境，DFT 用于设置对结构做 SCF 或 AIMD 所用环境。

```
{
    "DFT": {
        "command":"mpirun -np 4 PWmat",
        "task_run_num":1,
        "number_node": 1,
        "cpu_per_node": 4,
        "gpu_per_node": 4,
        "group_size": 1,
```

```
        "queue_name": "3080ti,3090",
        "custom_flags": [
        ],
        "_custom_flags": [
        ],
        "module_list": [
            "compiler/2022.0.2",
            "mkl/2022.0.2",
            "mpi/2021.5.1",
            "cuda/11.6",
            "pwmat"
        ]
    },

    "explore": {
        "command": "python sevennet_md.py",
        "group_size": 1,
        "number_node": 1,
        "gpu_per_node": 1,
        "cpu_per_node": 1,
        "queue_name": "3080ti",
        "custom_flags": [],
        "source_list": [
            "/share/app/anaconda3/envs/SevenNet/env.sh"
        ],
        "module_list": [
        ],
        "env_list": [
        ]
    },

    "direct": {
        "command": "python direct.py",
        "group_size": 1,
        "number_node": 1,
        "gpu_per_node": 1,
        "cpu_per_node": 1,
        "queue_name": "3080ti",
        "custom_flags": [],
        "source_list": [
            "/share/app/anaconda3/envs/m3gnet/env.sh"
        ],
        "module_list": [
```

```
        ],
        "env_list": [
        ]
    }
}
```

对于主动学习（run）可设置 `train`, `explore`, `direct`, DFT 四个模块，这里以 mcloud 环境设置为例。train 用于设置训练环境，explore 用于设置 lammps 分子动力学环境，direct 用于设置 direct 采样环境，DFT 用于设置对结构做 SCF 环境 或者 大模型 MD 环境。做：

```
{
  "train": {
    "command": "MatPL",
    "group_size": 1,
    "number_node": 1,
    "gpu_per_node": 1,
    "cpu_per_node": 1,
    "queue_name": "3080ti,3090",
    "custom_flags": [
    ],
    "source_list": [
      "/share/app/MATPL/MatPL-2025.3/env.sh"
    ],
    "module_list": [
    ],
    "env_list":[
    ]
  },
  "explore": {
    "command": "mpirun -np 1 lmp_mpi -in in.lammps",
    "group_size": 1,
    "number_node": 1,
    "gpu_per_node": 1,
    "cpu_per_node": 1,
    "queue_name": "3080ti,3090",
    "custom_flags": [],
    "source_list": [],
    "module_list": [
      "lammps4matpl/2025.3"
    ],
    "env_list":[]
  },
  "DFT": {
    "command": "mpirun -np 4 PWmat",
```

```
    "number_node": 1,
    "cpu_per_node": 4,
    "gpu_per_node": 4,
    "group_size": 1,
    "queue_name": "3080ti,3090",
    "custom_flags": [],
    "source_list": [],
    "module_list": [],
    "env_list":[
      "compiler/2022.0.2",
      "mkl/2022.0.2",
      "mpi/2021.5.1",
      "cuda/11.6",
      "pwmat"
    ]
  }
}
```

6.2 参数细节

参数可以分为 3 类。用于设置运行命令的 command；

用于设置每个计算任务的资源数量 number_node、cpu_per_node、gpu_per_node、group_size、queue_name、custom_flags；

用于加载软件和环境变量 custom_flags、source_list、module_list、env_list。

详细解释如下。### command

必选参数，设置模块对应命令。

不同任务的设置，例子：

对于 DFT 计算设置

```
PWmat 设置:
"command":"mpirun -np 4 PWmat"
VASP 设置:
"command":"vasp_std"
cp2k 设置:
"command":"mpirun -np $SLURM_NTASKS cp2k.popt"
```

对于 Lammps 计算设置：

```
"command":"mpirun -np 10 lmp_mpi"
```

-np 后面为使用的 CPU 数量，需要与`cpu_per_node`设置保持一致，如果`gpu_per_node`设置，那么 np 中设置的 cpu 任务将平均分配到 gpu_per_node 所设置的 GPU 上。

对于 MatPL 文档 模型训练设置：

```
"command":"MatPL"
```

6.2.1 group_size

该参数用于多个计算任务提交时的分组，同组内的计算任务将顺序执行。组间任务并行。

例如，对于 34 个 自洽计算任务，`"group_size":5`，此时将把 34 个自洽计算任务分为 6 个组，即 6 个 slurm 任务，每个 slrum 任务包含 5 个自洽计算（最后一个组有 4 个计算任务）。执行时，6 个 slurm 任务将同时提交到计算集群（slurm 任务内部的 5 个自洽计算将串行执行）。

- 对于训练或探索任务，默认值为 `1`，每组 1 个任务，即所有任务同时提交。
- 对于 AIMD 或 SCF 任务，即 `DFT` 下的参数，默认值为`-1`，所有任务将分到一个组内，执行时，每个任务串行执行（即任务执行完毕后再提交执行下一个任务）。建议您根据实际的任务数量设置该参数，避免同时提交大量 DFT 任务，挤占所有计算资源。

6.2.2 number_node

用于设置每个 slurm 任务的计算节点数量，默认值为 `1`，即 1 个计算节点。

在 train 模块，该参数自动设置 1。

6.2.3 gpu_per_node

用于设置每个节点下使用的 GPU 数目，默认值为 `0`，如果使用 PWMAT 做 DFT 计算（自洽计算、驰豫或者 AIMD）该值需要与 `"command"` 中设置的 GPU 数量一致。

在 train 模块，该参数自动设置 1。

6.2.4 cpu_per_node

用于设置每个节点使用的 CPU 数目，默认值为 `1`，注意该值需要“>= gpu_per_node”。

在 train 模块，该参数自动设置 1。

6.2.5 queue_name

必选参数，用于设置使用的计算机群分区，为 `","` 分割的字符串列表，例如 `"queue_name":"cpu, 3080ti, 3090"`。

6.2.6 custom_flags

用于设置 Slurm 脚本中的其他 #SBATCH 参数，可选参数，为 list 格式。例如，对于

```
"custom_flags": [
  "#SBATCH -x gn43,gn66"
]
```

在执行，将会把 `"#SBATCH -x gn43,gn66"` 自动拼接到 Slrum 脚本中。这里“#SBATCH”也可以省略，只写“-x gn43,gn66”。

例如对于设置：

```
"number_node": 1,
"cpu_per_node": 4,
"gpu_per_node": 4,
"queue_name": "3080ti,3090",
"custom_flags": [
  "#SBATCH -x gn43,gn66"
]
```

生成的 slurm 头文件为：

```
#SBATCH --nodes=1
#SBATCH --gres=gpu:4
#SBATCH --ntasks-per-node=4
#SBATCH --partition=3080ti,3090
#SBATCH -x gn43,gn66
```

由于部分机器不支持`--gpus-per-task`参数设置，因此 pwact 不会自动生成该行配置，如果需要设置，请写到 `custom_flags` 中即可。

6.2.7 source_list

用于设置 slurm 脚本在运行时需要导入的环境变量，可选参数，list 格式。例如，对于

```
"source_list": [
    "~/anaconda3/etc/profile.d/conda.sh"
]
```

在执行时，把字符串自动拼接前缀“source”后，将”source /opt/rh/devtoolset-8/enable”自动写入 slrum 脚本中。

6.2.8 module_list

用于设置 slurm 脚本在运行时需要加载的软件模块，可选参数，list 格式。

例如，对于

```
    "module_list": [
        "cuda/11.8",
        "intel/2020"
    ]
```

在执行时，将会把字符串“module load cuda/11.8”和“module load intel/2020”自动写入 slurm 脚本中。

6.2.9 env_list

用于设置 slurm 脚本在运行时需要加载的环境信息，可选参数，list 格式。

例如，对于

```
    "env_list": [
        "source the/path/MatPL-2025.3/env.sh"
    ]
```

在执行时，会将这条字符串完整写入 slurm 脚本中。

按照上述 queue_name、custom_flags、source_list、module_list、env_list 中的设置，生成的 slurm 脚本内容如下：

```
#SBATCH --partition=3080ti,3090
#SBATCH -x gn43,gn66

source /opt/rh/devtoolset-8/enable
module load cuda/11.8
module load intel/2020
source the/path/MatPL-2025.3/env.sh
```

6.3 配置案例详解

6.3.1 train 模块

对于 `train` 模块，需要加载 MatPL 的 Python 运行环境，如果使用 MCLOUD 上已安装的 MatPL 做训练，对应的设置如下：

```
  "train": {
    "command": "MatPL",
    "group_size": 1,
    "number_node": 1,
    "gpu_per_node": 1,
    "cpu_per_node": 1,
```

```
    "queue_name": "3080ti,3090",
    "custom_flags": [
    ],
    "source_list": [
    "/share/app/MATPL/MatPL-2025.3/env.sh"
    ],
    "env_list": [
    ],
    "module_list": [
    ]
  }
```

这里对于每个训练任务使用 1 个计算节点，使用该节点的 1 张 GPU,1 个 CPU，该节点位于分区 `3080ti` 或 `3090`。

如果从 MatPL 源码编译安装，对应的设置如下：

```
  "train":{
    "command": "MatPL",
    "group_size": 1,
    "number_node": 1,
    "gpu_per_node": 1,
    "cpu_per_node": 1,
    "queue_name": "3080ti,3090",
    "custom_flags": [
    ],
    "source_list": [
        "~/anaconda3/etc/profile.d/conda.sh",
        "the/path/MatPL-2025.3/env.sh"
    ],
    "env_list": [
        "conda activate matpl-2025.3"
    ],
    "module_list": [
        "cuda/11.8-share",
        "intel/2020"
    ]
  }
```

这里 `"~/anaconda3/etc/profile.d/conda.sh"` 为笔者计算集群中的 conda 加载路径，`matpl-2025.3` 为 MatPL 的 Python 环境，`the/path/MatPL-2025.3` 为源码所在路径。

6.3.2 explore 模块

对于 explore 模块，如果使用 MCLOUD 已安装的 LAMMPS 为例，直接加载 `lammps4matpl` 软件即可，完整的设置如下：

```
"explore": {
  "command": "mpirun -np 1 lmp_mpi",
  "group_size": 2,
  "number_node": 1,
  "gpu_per_node": 1,
  "cpu_per_node": 1,
  "queue_name": "3080ti,3090",
  "custom_flags": [
  ],
  "source_list": [

  ],
  "module_list": [
      "lammps4matpl/2025.3"
  ],
  "env_list":[

  ]
}
```

这里对于每个 lammps 任务，使用 1 个计算节点，使用该节点的 1 张 GPU,1 个 CPU，每 2 个 lammps 任务分为 1 个组。

如果从 LAMMPS 源码编译安装，对应的设置如下：

```
"explore": {
  "command": "mpirun -np 1 lmp_mpi",
  "group_size": 2,
  "number_node": 1,
  "gpu_per_node": 1,
  "cpu_per_node": 1,
  "queue_name": "3080ti,3090",
  "custom_flags": [
  ],
  "source_list": [
      "the/path/of/lammps/env.sh"
  ],
  "module_list": [
      "cuda/11.8-share",
      "intel/2020"
```

```
    ],
    "env_list":[
    ]
    }
```

这里 `the/path/of/lammps/env.sh` 为 lammps 源码所在路径，lammps 力场接口

6.3.3 DFT 模块

对于 DFT 模块，这里以加载 PWMAT 为例，设置如下。

```
"DFT": {
    "command":"PWmat",
    "number_node": 1,
    "cpu_per_node": 4,
    "gpu_per_node": 4,
    "group_size": 5,
    "queue_name": "3080ti,1080ti,3090",
    "custom_flags": [
    "#SBATCH -x gn18,gn17"
    ],
    "module_list": [
        "compiler/2022.0.2",
        "mkl/2022.0.2",
        "mpi/2021.5.1",
        "cuda/11.6"
    ],
    "env_list":[
    ]
}
```

这里对于每个 PWmat 任务，使用 1 个计算节点，使用该节点的 1 张 GPU,1 个 CPU，每 5 个 DFT 任务分为 1 个组。

7 Example for Si active learning

本案例为硅体系的主动学习过程，案例位于 `pwact/example/si_pwmat/`。对于 Mcloud 用户，请访问路径 `/share/public/PWMLFF_test_data/pwact_examples/25-pwact-demo` 即可。

案例首先通过 `INIT_BULK` 构造初始训练集，之后使用初始训练集训练模型，并使用在 INIT_BULK 中使用微扰产生的结构做为初始构型在 `300K`、`500K` 和 `900K` 做主动学习采样。

请注意，案例中提供的 DFT 设置仅用于程序执行流程测试，不保证计算精度。

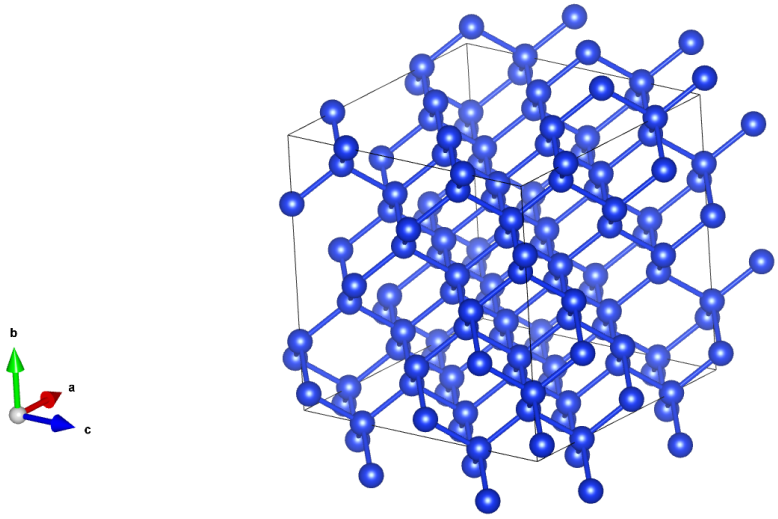

Figure 4: SI_POSCAR

7.1 预训练数据制备 INIT_BULK

7.1.1 启动命令

进入 `pwact/example/si_pwmat/init_bulk` 目录

```
pwact init_bulk init_param.json resource.json
```

7.1.2 INIT_BULK 目录结构

INIT_BULK的目录结构如下所示,`atom.config`、`POSCAR`、`resource.json`、`init_param.json`、`relax_etot.input`、`relax_etot1.input`、`aimd_etot1.input`、`aimd_etot2.input` 为输入文件，`collection` 为执行后的结果汇总目录。

7.1.3 collection 目录

`datapath.txt` 文件内容是预训练数据所在目录记录。

init_config_0 目录为 atom.config 进过驰豫、阔胞、缩放、微扰、aimd 后的结果汇总。

`relax.config` 是对 `atom.config` 做驰豫后得到的结构文件；`super_cell.config` 是对 `relax.config` 做 阔胞后得到结构文件；`0.9_scale.config` 和 `0.95_scale.config` 是对 `super_cell.config` 做晶格缩放后得到的结构文件；

`0.9_scale_pertub` 目录包括对 `0.9_scale.config` 结构做晶格和原子位置微扰后得到的 30 个结构；

train.xyz 目录是对微扰后的结构做按照 aimd_etot1.input 做 AIMD 之后，将得到轨迹提取成 extxyz 格式后的结果目录。如果是提取为 pwmlff/npy 格式，则为 PWdata 目录。内容如下所示。atom_type.npy 是结构的原子类型，position.npy 是结构中原子的位置信息，energies.npy、forces.npy、ei.npy、virials.npy 为结构的总能量、原子三个方向的力、原子能量和维里信息。ei.npy、virials.npy 是可选文件，如果轨迹中不包括原子能和维里，则不提取。

```
./init_config_0/PWdata/Si128
├── atom_type.npy
├── energies.npy
├── ei.npy
├── forces.npy
├── image_type.npy
├── lattice.npy
├── position.npy
└── virials.npy
```

7.1.3.1 目录

```
example/init_bulk
├──atom.config
├──POSCAR
├──resource.json
├──init_param.json
├──relax_etot.input
├──relax_etot1.input
├──relax_etot2.input
├──aimd_etot1.input
├──aimd_etot2.input
└──collection
    ├──datapath.txt
    ├──init_config_0
    │   ├──super_cell.config
    │   ├──0.9_scale.config
    │   ├──0.9_scale_pertub
    │   │       ├──0.9_scale.config
    │   │       ├──0_pertub.config
    │   │       ├──1_pertub.config
    │   │       ├──2_pertub.config
    │   │       ...
    │   │       └──30_pertub.config
```

```
│   ├──0.95_scale.config
│   ├──0.95_scale_pertub
│   │       ├──0.95_scale.config
│   │       ├──0_pertub.config
│   │       ├──1_pertub.config
│   │       ├──2_pertub.config
│   │       ...
│   │       └──30_pertub.config
│   └──train.xyz
├──init_config_1
...
```

7.2 主动学习 run

我们使用 INIT_BULK 案例中的预训练数据和微扰后的结构，在 500K、800K 和 1100K 下做主动学习。

7.2.1 启动命令

执行完毕 init_bulk 命令之后，进入 `pwact/example/si_pwmat/` 目录：

```
pwact run param.json resource.json
```

7.2.2 主动学习文件目录

主动学习的目录结构如下所示。

`param.json` 和 `resource.json` 为主动学习输入控制文件，`scf_etot.input` 为主动学习做自洽计算的输入文件。

`si.al` 为主动学习流程记录文件，记录已执行的主动学习流程。

`iter_result.txt` 为主动学习每个轮次中探索和选取结构用于标注的记录。内容如下例所示。第一行为本轮次的选点情况。Total structures 为采集到的结构数量 = ccurate 准确点 + selected 候选点数量 + error 错误点数量。第二行为 MD 轨迹的总结，以例子中所示，表示共运行了 16 条 MD 轨迹，其中 16 条 MD 正确运行结束，0 条轨迹运行失败。运行失败的轨迹汇总在 error_traj.log 文件中。运行失败的情况一般是力场不够准确，导致 MD 运行崩溃。接下来的行为标记过程中，已收敛和未收敛的结构数量统计以及未收敛的结构路径，对于为收敛的结构，在标记结束后，不会加入到训练集中。

```
iter.0000  Total structures 1296    accurate 6 rate 0.46%    selected
 ↪  23 rate 1.77%    error 1267 rate 97.76%
A total of 16 MD trajectories were run. with 16 trajectories
 ↪  correctly executed and 0 trajectories normally completed.
```

```
For detailed information, refer to File error_traj.log.

Number of converged files: 194, number of non-converged files: 3
List of non-converged files:
/share/public/PWMLFF_test_data/pwact_examples/25-pwact-
 ↪  demo/auag_vasp/run_iter_lmps/iter.0000/temp_run_iter_work/02.label/scf/md.00
 ↪  scf/OUTCAR
...

iter.0001  Total structures 12816    accurate 598 rate 4.67%
 ↪  selected 2926 rate 22.83%    error 9292 rate 72.50%
A total of 16 MD trajectories were run. with 16 trajectories
 ↪  correctly executed and 0 trajectories normally completed.
For detailed information, refer to File error_traj.log.

Number of converged files: 194, number of non-converged files: 3
List of non-converged files:
/share/public/PWMLFF_test_data/pwact_examples/25-pwact-
 ↪  demo/auag_vasp/run_iter_lmps/iter.0001/temp_run_iter_work/02.label/scf/md.00
 ↪  scf/OUTCAR
...
```

`iter.0000` 为第一轮次的主动学习目录，`iter.0001` 为第二轮次的主动学习目录，以此类推。

`00.train`、`01.explore`、`02.label` 为主动学习轮次中对应的模型训练、探索、标记三个任务所在目录。

7.2.3 00.train 目录

对于 `00.train` 目录，这里采用 4 模型的委员会查询策略，训练 4 个模型，这 4 个模型只有网络参数的初始化值不同，其他完全相同。`0-train.job`、`1-train.job`、`2-train.job`、`3-train.job` 为执行训练任务的 4 个 slurm 任务脚本。训练任务执行完毕后，将生成 4 个标识任务执行成功的 tag 文件 (`0-tag.train.success`、`1-tag.train.success`、`2-tag.train.success`、`3-tag.train.success`)，以及 4 个模型，目录为 `train.000`、`train.001`、`train.002`、`train.003`。

以 `train.000` 目录为例，`train.json` 为 MatPL 模型训练的输入文件。`std_input.json` 为 MatPL 输出的训练参数设置汇总。`model_record` 为模型的保存目录，`dp_model.ckpt` 为模型文件，`epoch_train.dat` 为在每个 epoch 下的平均训练误差，为每个 epoch 训练结束后在验证集上的平均误差，保存在 `epoch_valid.dat`。

`torch_script_module.pt` 为训练结束后，使用 jitscript 工具编译 dp_model.ckpt 后的模型文件，做为力场，用于接下来在 lammps 中的模拟。

7.2.4 01.explore 目录

`01.explore` 目录包括两个子目录，`md` 和 `select` 子目录。

`md` 目录为主动学习的探索目录，包括在不同温度、压强等条件下对初始结构调用 MatPL 力场做分子动力学模拟的文件（输入文件、轨迹）。

之后把模拟得到的结构（轨迹）根据委员会查询方法的上下界设置做筛选，筛选的结果保存在 `select` 子目录。

7.2.4.1 md 子目录 md 子目录包括两个子目录,目录名称为 `md.***.sys.***/md.***.sys.***.t.***`，例如 `md.000.sys.000/md.000.sys.000.t.000`，对于 `md.000.sys.000` 目录，这里 `md.000` 的 000 指 `param.json` 中的`md_jobs` 对第 0 个 `md` 设置；sys.000 为`sys_index` 的下标 0 对应的结构。包括 `md.000.sys.000.t.000`、`md.000.sys.000.t.001` 两个子目录，分别表示在温度 `temps` 对应下标为 `0` 和 `1` 温度下的分子动力学模拟。

在每个 `md.*.sys.*` 目录下都有一个 `model_devi_distribution.png` 文件，是对该目录下所有轨迹的偏差值分布统计绘图。

在最后一级子目录下，是对应 MD 设置以及轨迹文件：

```
0_torch_script_module.pt, 1_torch_script_module.pt,
 ↪  2_torch_script_module.pt, 3_torch_script_module.pt,
 ↪  atom_type.txt, in.lammps, lmp.config, log.lammps, md.log,
 ↪  model_devi.out, tag.md.success

model_devi.out 格式:
#    step       avg_devi_f       min_devi_f       max_devi_f
 ↪  avg_devi_e       min_devi_e       max_devi_e
       0      0.008682422      0.000574555      0.014187865
 ↪  0.030333196      0.011702489      0.043806718
      10      0.016611307      0.000627126      0.027185410
 ↪  0.031661788      0.005340899      0.048241921
      20      0.028689107      0.000811486      0.040269587
 ↪  0.036761117      0.000214928      0.059791462
      30      0.046457606      0.001416745      0.063635973
 ↪  0.049553956      0.000165127      0.081012839
      40      0.065051169      0.000681319      0.089395358
 ↪  0.065790218      0.000036582      0.103522832
```

model_devi.out 的第 1 列为当前步编号，第 2 列为力偏差的均值，第 3 列为力偏差最小值，第 4 列为力偏差最大值。力偏差最大值计算公式如下所示：

$$\varepsilon_t = \max_i \sqrt{\left\langle \sum_{w=1}^{W} \|F_{w,i}(R_t) - \hat{F}_i\|^2 \right\rangle}, \quad \hat{F}_i = \frac{1}{W}\sum_{w=1}^{W} F_{w,i}$$

这里 W 为模型数量，i 为原子下标，$<>$ 为取平均。

后三列分别是原子能量的偏差均值、最小值和最大值。在 PWact 主动学习中使用的是第 4 列的最大力偏差值。

7.2.4.2 select 子目录 以下 `.csv` 文件内容包含 3 列，分别是力偏差（`devi_force`）、结构在轨迹中对应的编号（`config_index`）、轨迹的文件路径（`file_path`）。

`accurate.csv` 是力偏差小于设置的力偏差下限的结构汇总。

`fail.csv` 是力偏差大于设置的力偏差上限的结构。

如果候选的结构（力偏差介于设置的力偏差上下限之间的结构）超过设置的最大选点数量 `max_select`，则将候选的结构随机选取 `max_select` 个，存入 `candidate.csv` 文件，其余的存入 `candidate_delete.csv` 文件。

`model_devi_distribution-md.*.sys.*.png` 为超链接文件，是探索结构的偏差值分布绘图。

`error_traj.log` 是 MD 过程为执行 MD 过程中断的轨迹，中断的原因一般是力场不准确造成原子丢失、原子靠的太近等。

`select_summary.txt` 是筛选点的数据量信息汇总，内容如下例所示。

```
Total structures 862   accurate 0 rate 0.00%   selected 616 rate 71.46%   err

Select by model deviation force:
Accurate configurations: 0, details in file accurate.csv
Candidate configurations: 616, randomly select 27, delete 589
        Select details in file candidate.csv
        Delete details in file candidate_delete.csv.
Error configurations: 246, details in file fail.csv

A total of 10 MD trajectories were run. with 8 trajectories correctly execute
For detailed information, refer to File error_traj.log.
```

7.2.5 02.label 目录

`02.label` 目录包括 `scf` 和 `result` 两个子目录。`scf` 为对 `explore` 中筛选出的点做自洽计算的目录。自洽计算后，将结构和对应的能量、力、原子能、维里提取为 pwdata 格式，保存在 `result` 子目录，作为后面主动学习轮次的训练数据。

7.2.5.1 scf scf 下的一级和二级子目录 `md.*.sys.*/md.*.sys.*.t.*` 与 md 子目录的结构和名称意义完全相同。在二级子目录下，是自洽计算的目录。以 `scf/md.000.sys.000/md.000.sys.000.t.00` `scf` 目录为例，这里 820 指 `md.000.sys.000/md.000.sys.000.t.000` 轨迹的第 820 步对应的结构。该目录下，`820.config` 为自洽计算的输入结构，`etot.input` 为自洽计算的输入控制文件，`REOPORT` 和 `OUT.MLMD` 为 PWMAT 的输出文件。其中 `OUT.MLMD` 文件包括了自洽计算后的结构原子位置信息、能量、力等信息。

7.2.5.2 result `result` 为标记结束后的带标签数据集汇总，如果设置 `data_format` 为 `extxyz` 格式，将提取为 train.xyz 文件。如果为 `pwmlff/npy` 格式，则为 PWdata 目录，目录下是具体数据。

7.2.5.3 目录

```
example
├──param.json
├──resource.json
├──scf_etot.input
├──si.al
├──iter_result.txt
├──iter.0000
│     ├──00.train
│     │    ├──0-train.job
│     │    ├──1-train.job
│     │    ├──3-train.job
│     │    ├──2-train.job
│     │    ├──train.000
│     │    │    ├──train.json
│     │    │    ├──std_input.json
│     │    │    ├──model_record
│     │    │    │    ├──dp_model.ckpt
│     │    │    │    ├──epoch_train.dat
│     │    │    │    └──epoch_valid.dat
│     │    │    ├──torch_script_module.pt
│     │    │    └──tag.train.success
│     │    ├──train.001
│     │    │    └──...
│     │    ├──train.002
│     │    │    └──...
│     │    ├──train.003
│     │    │    └──...
│     │    ├──0-tag.train.success
```

```
│   │   ├──1-tag.train.success
│   │   ├──2-tag.train.success
│   │   └──3-tag.train.success
│   ├──01.explore
│   │   ├──md
│   │   │   ├──md.000.sys.000
│   │   │   │   ├──md.000.sys.000.t.000.p.000
│   │   │   │   ├──...
│   │   │   │   └──model_devi_distribution.png
│   │   │   ├──md.000.sys.001
│   │   │   ├──md.001.sys.000
│   │   │   └──md.001.sys.003
│   │   └──select
│   │       ├──accurate.csv
│   │       ├──candidate.csv
│   │       ├──candidate_delete.csv
│   │       ├──fail.csv
│   │       ├──error_traj.log
│   │       ├──select_summary.txt
│   │       ├──model_devi_distribution-md.000.sys.000.png
│   │       └──...
│   └──02.label
│   │   ├──scf
│   │   │   ├──md.000.sys.000
│   │   │   │   ├──md.000.sys.000.t.001
│   │   │   │   │   ├──820-scf
│   │   │   │   │   │   ├──820.config
│   │   │   │   │   │   ├──etot.input
│   │   │   │   │   │   ├──REPORT
│   │   │   │   │   │   └──OUT.MLMD
│   │   │   │   │   ├──200-scf
│   │   │   │   │   └──...
│   │   │   │   └──md.000.sys.000.t.000
│   │   │   │       └──...
│   │   │   ├──md.000.sys.001
│   │   │   ├──...
│   │   │   ├──md.001.sys.000
│   │   └──result
│   │       └──train.xyz
```

```
├──iter.0001
│     └──...
├──...
```

8 Example for Si active learning (Bigmodel and Direct)

本案例为硅体系的主动学习过程，采用了大模型（eqv2）和 direct 采样。在 `PWact/examples/` 下准备了多种组合设置，每种组合的信息请参考`PWact/examples/README.md`。对于 Mcloud 用户，请访问路径`/share/public/PWMLFF_test_data/pwact_examples/25-pwact-demo` 即可。这里以 [`PWact/examples/si_pwmatgaussion_bigmodel_direct`] 为例介绍大模型和 direct 方法的参数设置、脚本设置。对于 `INIT_BULK` 构造初始训练集，以`examples/si_pwmatgaussion_bigmodel_direct/init_bulk_bigmodel`为例：

- step1. 用 PWmat (gaussion 基组) 做 relax；
- step2. 对结构调用大模型（seventnet）做分子动力学；
- step3. 对分子动力学得到的轨迹做 direct 采样，去掉轨迹中相似的结构，筛选出的结构用于后续主动学习

请注意，本目录中使用的是 gaussian 基组的 PWmat，仅用于快速测试，例子生成的数据不具有可靠性。

8.1 预训练数据制备 INIT_BULK

8.1.1 启动命令

进入 `examples/si_pwmatgaussion_bigmodel_direct/init_bulk_bigmodel` 目录，这里提供了在 mcloud 上的执行脚本，也可以通过如下命令执行。

```
pwact init_bulk init_param.json resource.json
```

8.1.2 INIT_BULK 目录结构

INIT_BULK 目录与 si_pwmat 案例 相似，只是多了一项 `bigmodel` 目录，如下所示。

```
.
├── datapath.txt
├── init_config_0/
├── init_config_1/
└── bigmodel/
    ├── 0-bigmodel.job
```

```
├── 0-tag.bigmodel.success
├── 1-bigmodel.job
├── 1-tag.bigmodel.success
├── ...
├── direct/
│   ├── 0-direct.job
│   ├── 0-tag.direct.success
│   ├── candidate.json
│   ├── candidate.xyz
│   ├── Cov_score.png
│   ├── direct.py
│   ├── PCA_direct.png
│   ├── PCA_variance.png
│   ├── PWdata
│   ├── select_idx.dat
│   ├── select.xyz
│   └── tag.direct.success
├── init_config_0/
│   ├── 0.95_scale/
│   │   ├── 3_bigmodel/
│   │   │   ├── npt.log
│   │   │   ├── POSCAR
│   │   │   ├── sevennet_md.py
│   │   │   ├── tag.bigmodel.success
│   │   │   ├── tmp.traj
│   │   │   └── traj.xyz
│   │   ├── 4_bigmodel/
│   │   └── 5_bigmodel/
│   ├── 0.9_scale/
│   └── 1.0_scale/
└── init_config_*/
```

- `*-bigmodel.job` 和 `*-tag.bigmodel.success` 为 大模型运行 MD 的 slurm 脚本和执行成功的 tag 标记文件
- `init_config_0`、`init_config_1`、...目录为执行大模型 MD 的工作目录
- `direct` 目录为执行 `direct` 方法筛选结构的工作目录

8.1.3 接入 大模型 MD

对于大模型 MD，提供了用户自定义接口，要求用户配置大模型的运行环境，可参考例子中使用的sevennet_md.py。

如目录 `bigmodel/init_config_0/0.95_scale/3_bigmodel/` 所示，在 init_bulk 运行过程中，pwact 检查到存在 bigmodel 设置时，会把对应结构按照类似 `bigmodel/init_config_0/0.95_sca` 录所示生成，包括一个 VASP/POSCAR 格式的 POSCAR 文件，拷贝用户提供的运行 MD 脚本的接口文件到该目录，如 sevennet_md.py 文件，根据用户在 resource.json 中设置的 `explore`，自动生成 slurm 脚本，提交运行。

用户需要在脚本中读取 POSCAR 文件后运行 MD，将生成的轨迹转换为 `extxyz` 格式的文件，文件名称固定为 `traj.xyz`。之后，pwact 将检测该文件，用于后续处理。

8.1.4 接入 direct 采样

对于 direct 采样，提供了用户自定义接口，要求用户 direct 方法的运行环境，可参考例子中使用的direct.py。

如目录 `bigmodel/direct/` 所示，在 大模型 MD 运行结束后，会生成多条轨迹文件，pwact 会自动探测轨迹文件，将文件合并为 一个 名为 `candidate.xyz` 的文件，放到目录 `bigmodel/direct` 下。拷贝用户在 direct_input 提供的运行 direct 脚本的接口文件到该目录，如 direct.py 文件，根据用户在 resource.json 中设置的 `direct`，自动生成 slurm 脚本，提交运行。

用户需要在脚本中读取 `candidate.xyz` 文件，之后需要生成 `select_idx.dat` 和 `select.xyz` 两个文件，其中 select_idx.dat 保存筛选出的结构在 candidate.xyz 中对应的下标，select.xyz 保存筛选出的结构。pwact 会自动读取这两个文件用于后续的执行。

8.2 主动学习 run

我们使用 INIT_BULK 案例中的预训练数据和微扰后的结构，在 500K、800K 和 1100K 下做主动学习。

启动命令：执行完毕 init_bulk 命令之后，进入 `examples/si_pwmatgaussion_bigmodel_direct/run_` 目录：

```
pwact run param.json resource.json
```

8.2.1 主动学习文件目录

主动学习的目录结构与 si_pwmat 例子中相似。

8.2.2 train 目录

train 目录 与 si_pwmat 例子 相同

8.2.3 explore 目录

`explore` 目录除了 `md` 和 `select` 子目录外，新增加了一个名为 `bigmodel` 的子目录，子目录内容与 init_bulk direct 中相同。

md 和 `select` 子目录与 si_pwmat 例子 相同

8.2.4 label 目录

如果设置了使用大模型做标注`bigmodel_script`，则在 label 目录下新增一个 `bigmodel` 子目录。该子目录结构如下所示：

```
bigmodel/
├── 0-bigmodel.job
├── eqv2_label.py
├── select.xyz
└── train.xyz
```

- `eqv2_label.py` 为 用户设置的 bigmodel_script 脚本文件，pwact 运行时会将改文件拷贝到 bigmodel 目录下
- `select.xyz` 为 大模型标注的输入结构文件，格式为 extxyz
- `train.xyz` 为 大模型标注后的输出结构文件，格式为 extxyz，相比于 select.xyz，结构顺序相同，但是多了 能量和力的信息
- `0-bigmodel.job` 为 pwact 根据用户在 resource.json 中的 DFT 设置，生成的 slurm 脚本

8.2.5 接入 大模型 标记

对于 大模型 标记，提供了用户自定义接口，要求用户提供 大模型 的运行环境，可参考例子中使用的eqv2_label.py。

如目录 `bigmodel/` 所示，在 explore 步骤，经过多模型偏差筛选（或继续使用 direct 筛选）后，pwact 会自动探测筛选出的结构，将文件合并为 一个 名为 `select.xyz` 的文件，放到目录 `bigmodel/` 下。拷贝用户在 bigmodel_script 提供的运行 bigmodel 脚本的接口文件到该目录，如 eqv2_label.py 文件，根据用户在 resource.json 中设置的 DFT，自动生成 slurm 脚本，提交运行。

用户需要在脚本中读取 `select.xyz` 文件，之后需要生成 `train.xyz` 文件，train.xyz 相比于 select.xyz 多了能量和力信息。pwact 会自动读取这该文件用于后续的执行。

8.2.6 result

`result` 为标记结束后的带标签数据集汇总，如果设置 `data_format` 为 `extxyz` 格式，将提取为 train.xyz 文件。如果为 pwmlff/npy 格式，则为 PWdata 目录，目录下是具体数据。

9 Example for AuAg active learning

本案例为金银合金体系的主动学习过程，案例位于 `pwact/example/auag_pwmat/` 首先通过 `INIT_BULK` 构造初始训练集，之后使用初始训练集训练模型，使用在 INIT_BULK 中使用微扰产生的结构做为初始构型，使用用户输入的 `lammp.in` 输入控制文件做主动学习采样。:::tip 使用用户输入的 `lammp.in` 输入控制文件下做主动学习采样在 `>= pwact-0.4` 版本中开始支持。:::

以下案例使用的 DFT 计算软件为 PWMAT,案例也提供了使用 VASP 的对应设置`pwact/example/auag_vasp/`、CP2K 的对应设置`pwact/example/auag_cp2k/`。对于 Mcloud 用户，请访问路径`/share/public/PWMLFF_test_data/pwact_examples/25-pwact-demo` 即可。

请注意，案例中提供的 DFT 设置仅用于程序执行流程测试，不保证计算精度!

9.1 预训练数据制备 INIT_BULK

9.1.1 启动命令

进入 `pwact/example/auag_pwmat/init_bulk` 目录

```
pwact init_bulk init_param.json resource.json
```

9.1.2 INIT_BULK 目录结构

INIT_BULK 目录与 si_pwmat 案例 目录结构相同。

9.2 主动学习 run

执行完毕 init_bulk 命令之后，进入 `examples/auag_pwmat/run_iter_lmps` 目录：启动命令:

```
pwact run param.json resource.json
```

9.2.1 主动学习文件目录

主动学习目录结构与 si_pwmat 例子目录结构相同。唯一区别是对 md 的目录名称，如'md.000.sys.000/md.000.sys.000.t.000.p.000'将变成'md.000.sys.000/md.000.sys.000.lmps.000'，这里 `lmps.000` 用于标示使用的 lammps.in 文件编号。

```
example
├──param.json
├──resource.json
├──scf_etot.input
├──iter.0000
│     ├──00.train
│     │    ├──...
...
│     ├──01.explore
│     │    ├──md
│     │    │     ├──md.000.sys.000
│     │    │     │     ├──md.000.sys.000.lmps.000
│     │    │     │     ├──...
│     │    │     │     └──model_devi_distribution.png
│     │    │     ├──md.000.sys.001
│     │    │     ├──md.001.sys.000
│     │    │     └──md.001.sys.003
│     │    └──select
│     │         ├──accurate.csv
│     │         ├──candidate.csv
│     │         ├──candidate_delete.csv
│     │         ├──fail.csv
│     │         ├──error_traj.log
│     │         ├──select_summary.txt
│     │         ├──model_devi_distribution-md.000.sys.000.png
│     │         └──...
│     └──02.label
│     │    ├──scf
│     │    │     ├──md.000.sys.000
│     │    │     │     ├──md.000.sys.000.t.001
│     │    │     │     │     ├──820-scf
│     │    │     │     │     │     ├──820.config
│     │    │     │     │     │     ├──etot.input
│     │    │     │     │     │     ├──REPORT
│     │    │     │     │     │     └──OUT.MLMD
│     │    │     │     │     ├──200-scf
│     │    │     │     │     └──...
│     │    │     │     └──md.000.sys.000.t.000
│     │    │     │           └──...
│     │    │     ├──md.000.sys.001
│     │    │     ├──...
│     │    │     ├──md.001.sys.000
│     │    └──result
│     │         └──train.xyz
```

```
├──iter.0001
│     └──...
├──...
```